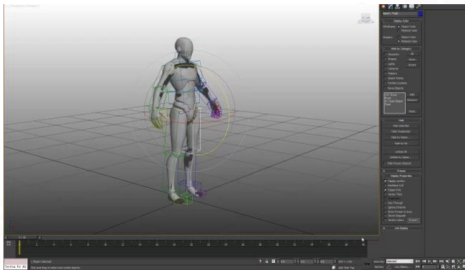




Animation System

Advanced Animation Technology

How to Achieve the Animation Effect in Real Game?



How?



simple animation



complex animation in real game



Animation Blending

- The term *animation blending* refers to any technique that allows more than one animation clip to contribute to the final pose of the character

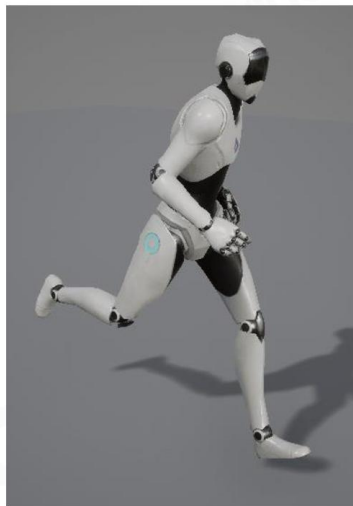


Case: Walking to Running

- Assume the character walks at 1.5m/s and runs at 3.0m/s in our game
- As the character's speed increase, we want to switch its animation from walking to running



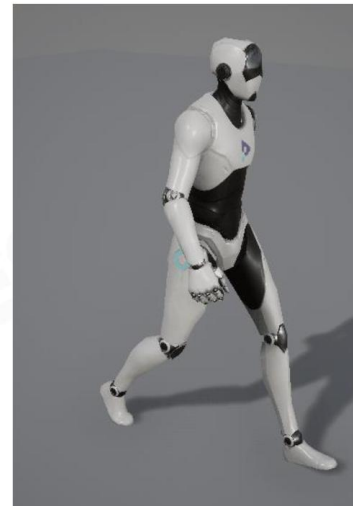
walking clip



running clip



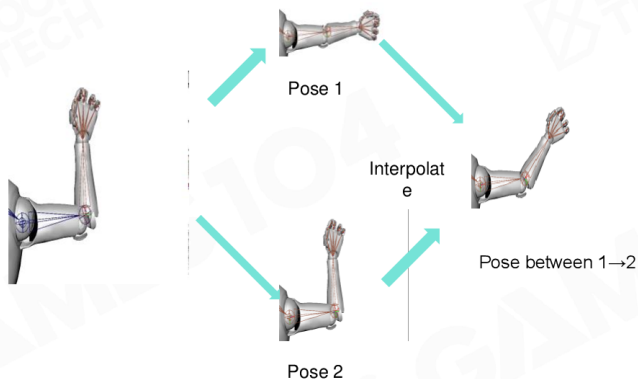
No blending



Blend Result

Math of Blending: LERP Linear interpolation: 가장 쉬운 보간

Use LERP to get intermediate frame from poses of different clips
Weight is controlled by game parameters, i.e, character speed



LERP between key frames in one clip



Blending is LERP between poses of different clips



Calculate Blend Weight

$speed_{current}$: current speed
 $speed_1$: speed of clip1
 $speed_2$: speed of clip2
 $weight_1$: calculated weight of clip1
 $weight_2$: calculated weight of clip2

$$weight_1 = \frac{speed_{current} - speed_2}{speed_1 - speed_2}$$

$$weight_2 = \frac{speed_{current} - speed_1}{speed_2 - speed_1}$$

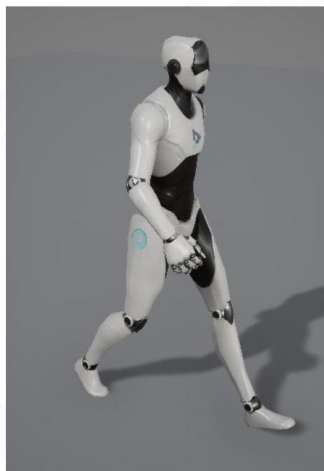


Blending is LERP between poses for different clips

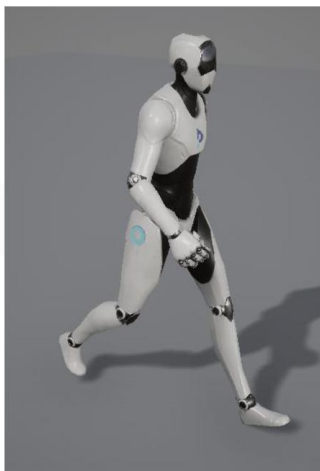


Case: Walking to Running

In order to achieve the desired effect, we need a lot of animation clips with intermediate speeds. Let the animators produce a whole bunch?



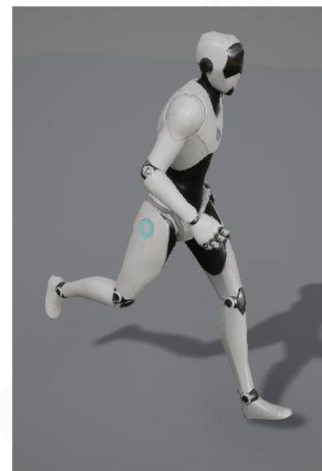
1.5m/s



1.9m/s



2.3m/s



2.7m/s



3.0m/s



Blend Space



1D Blend Space: Directional Movement

Players can move forward from multiple angles

We can blend any angle from three clips:

- Strafe Left clip
- Run Forward clip
- Strafe Right clip

The technique is called 1D Blend Space.



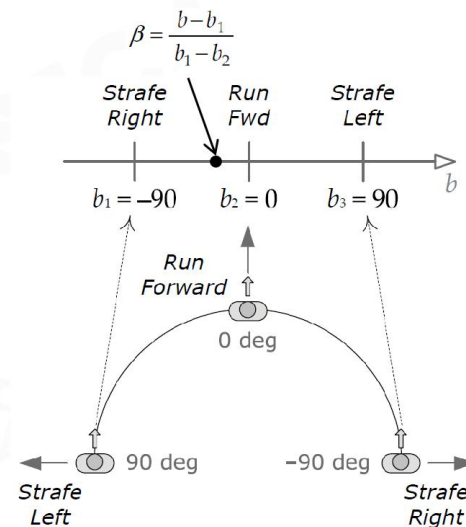
leftward



forward



rightward

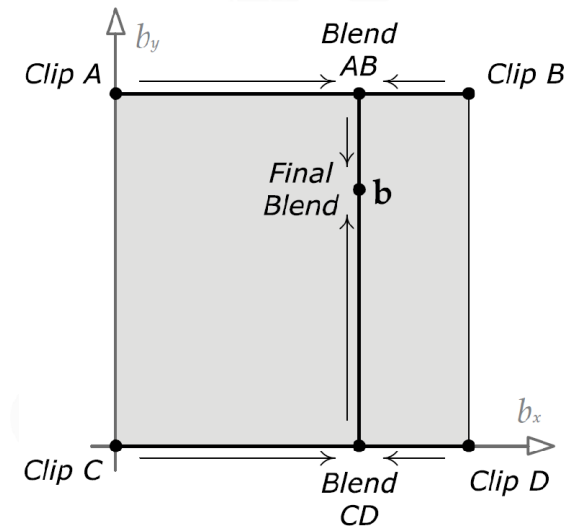


multi-directional movement

Directional Walking and Running

Players can change direction and speed at the same time

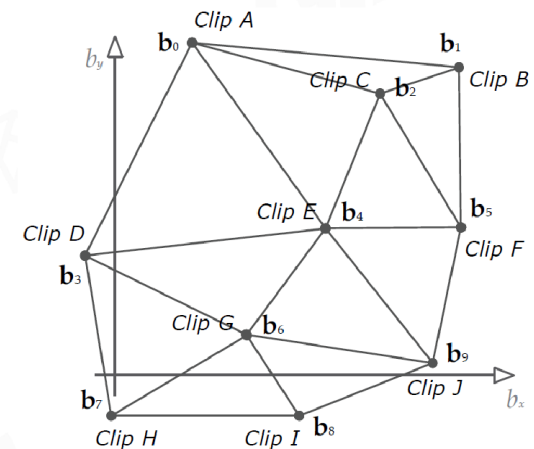
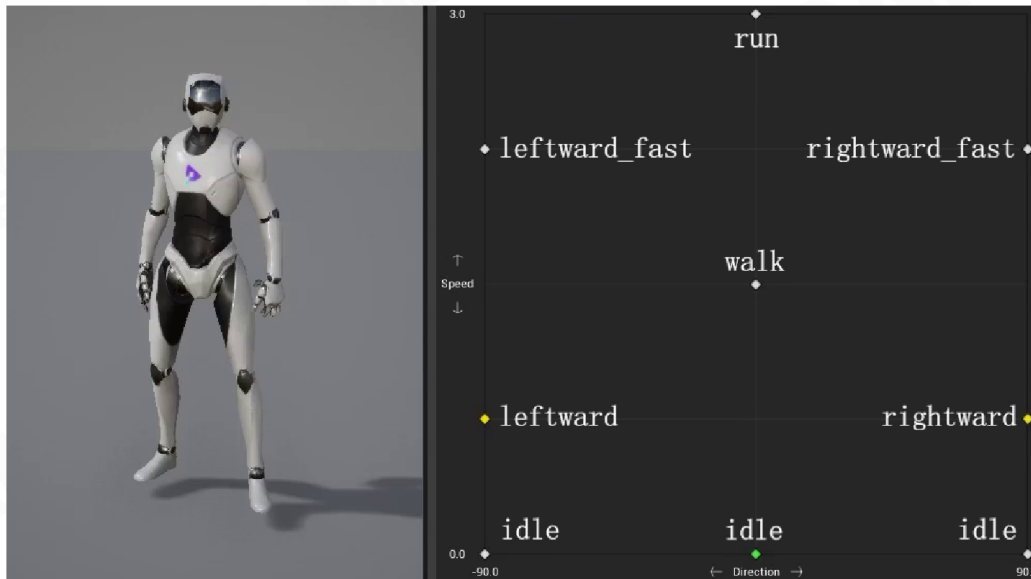
We simply place the two 1D Blend Spaces orthogonally and we get an 2D Blend Space



쌍선형 보간 = bilinear interpolation

2D Blend Space

Since the movement speed in the lateral direction is lower in the forward direction, the character should enter the running state in a lower speed in the lateral direction



Delaunay Triangulation

삼각형 내부의 보간: barycentric interpolation

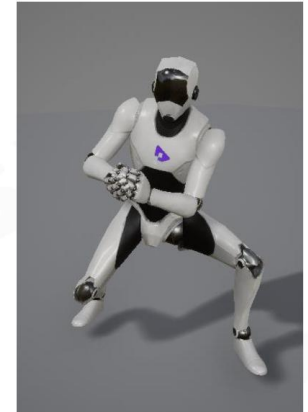
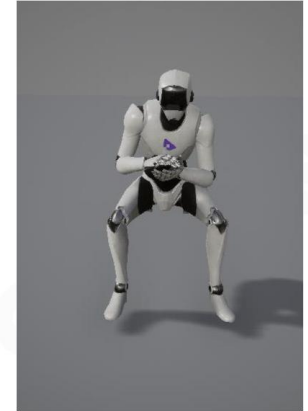
Case: Applauding on Different Poses

There are multiple robots in different poses in the scene

We need to make applause animations for various poses separately

Is it possible to make a single applauding animation that can be applied to all poses?

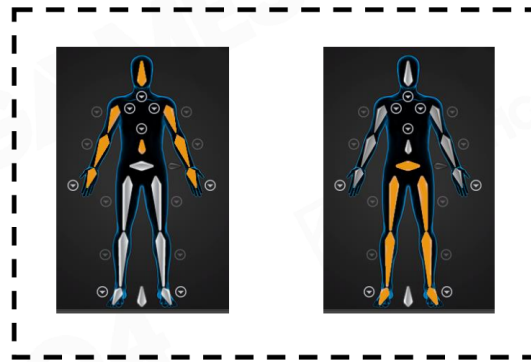
단순 보간으로는 어려울 수 있다.
손뼉만 섞여야 하므로...



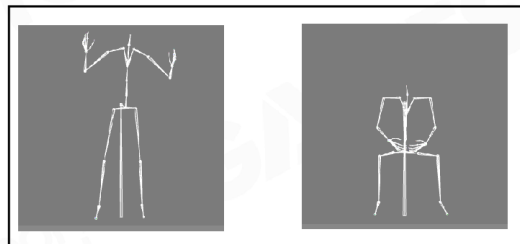


Skeleton Masked Blending

The set of all blend percentages for the entire skeleton $\{\beta_j\}_{j=0}^{N-1}$ is sometimes called a blend mask b

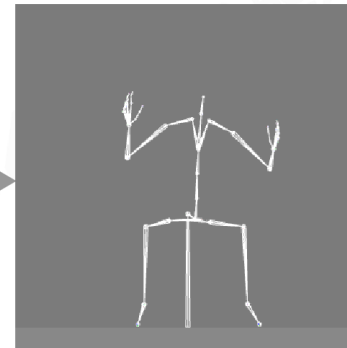


Blend Mask



standing clapping

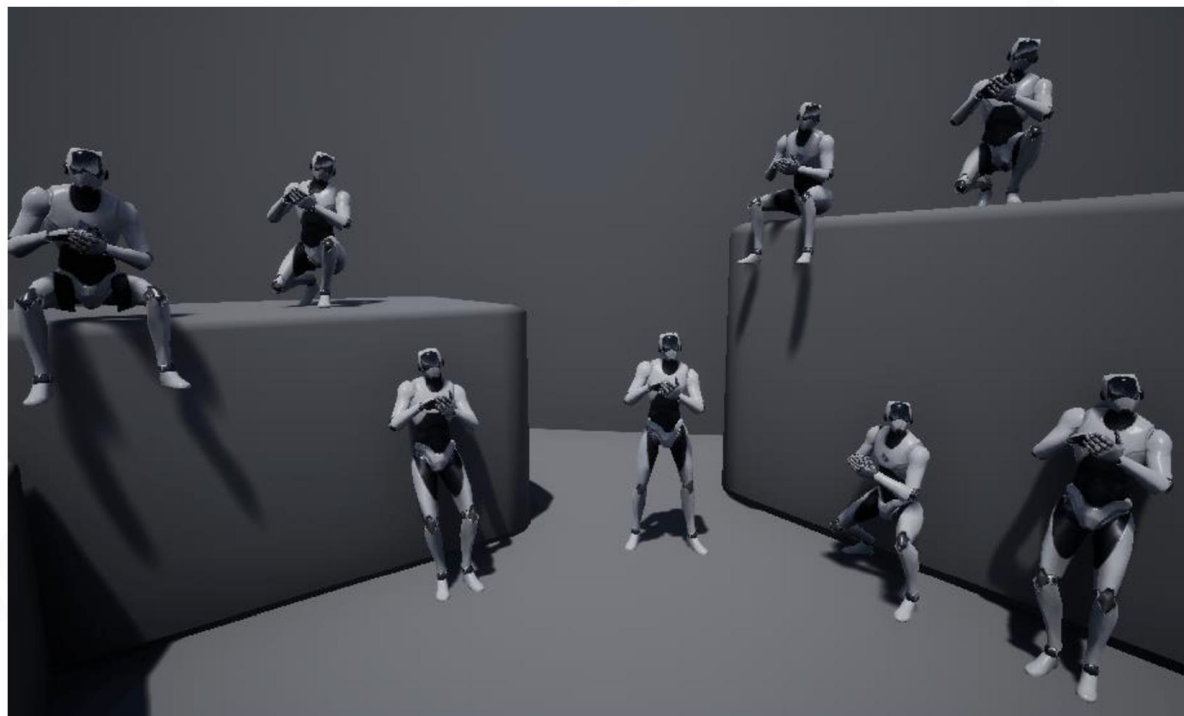
sitting



sitting clapping



Case: Warm Welcome from the Robots



We will let robots **applauding** in different poses



Additive Blending

Add a difference clip into a regular clip to produce a new clip

Additive Blending introduces a new kind of animation called a ***difference clip***, which represents the difference between two regular animation clips.

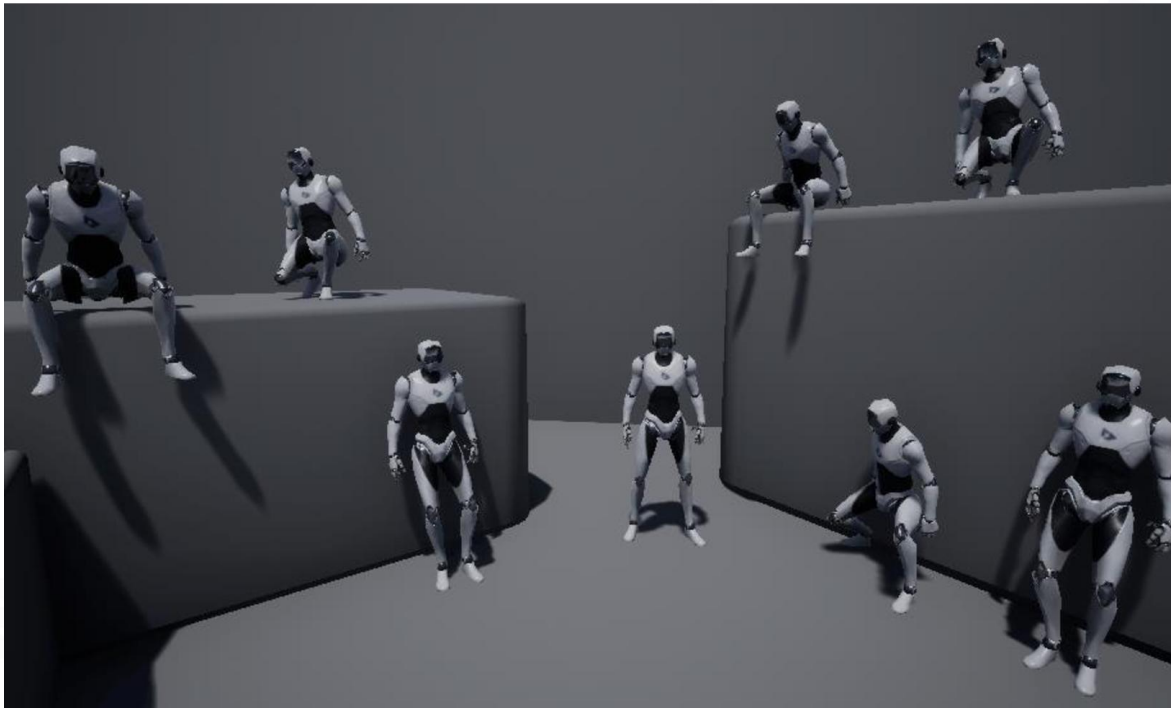
A difference clip can be added into a regular animation clip in order to produce interesting variations in the pose and movement of the character.



애니메이션 블렌딩 에서 Additive Blending은 기본 자세(base pose)에 추가 동작(additive animation)을 더해서 자연스러운 보조 움직임을 만드는 기법:
이 추가동작은 difference 클립으로 저장해 둘 수 있다.



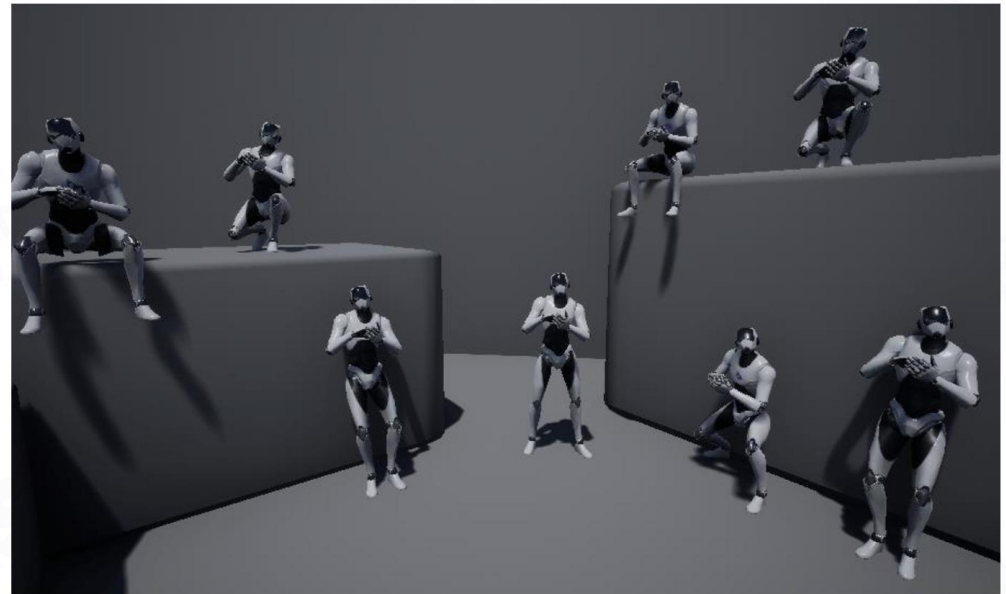
Nodding to Camera





Animation Blending Summary

- 1D Blend Space
 - Blend poses based on a single input value
- 2D Blend Space
 - Blend poses based on two input values
 - Triangular blend
- Masked Blending
- Additive Blending





Action State Machine (ASM)

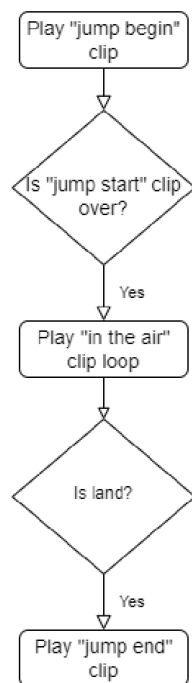


Case: Jumping

How to animate jump?

Blend Space is synchronous,
but jump is stateful

We usually model the jumping
action via a finite state machine,
commonly known as the Action
State Machine (ASM)





ASM Definition

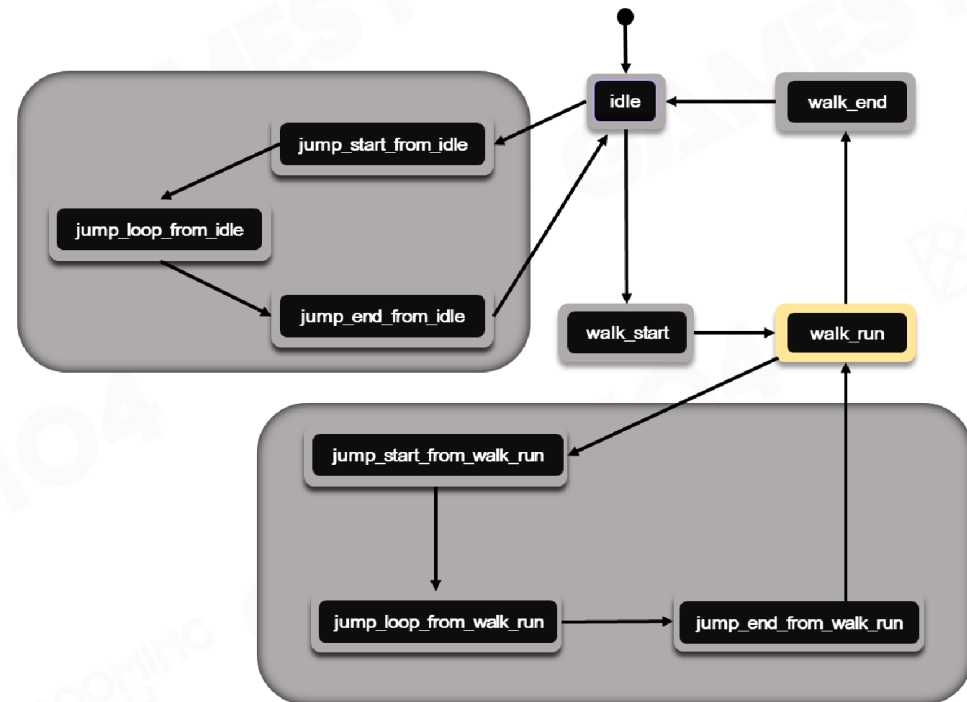
- ASM consists of nodes and transitions
- Node types
 - Blend space
 - Clip

```
class ActionStateMachineClipNode
```

```
{  
    AnimationClip m_clip;  
    bool m_is_loop;  
};
```

```
class ActionStateMachineBlendSpaceNode
```

```
{  
    BlendSpace m_blend_space;  
    bool m_is_loop;  
};
```





ASM Definition

- Transition type
 - simply “pop” from one state to another
 - cross-fade from one state to the next
 - Special transitional states

```
class ActionStateMachineTransition
{
    int    m_source_node_index;
    int    m_target_node_index;
};
```

```
class ActionStateMachineTransitionWithCrossFade
{
    int    m_source_node_index;
    int    m_target_node_index;
    float  m_duration;
    Curve  m_curve;
};
```

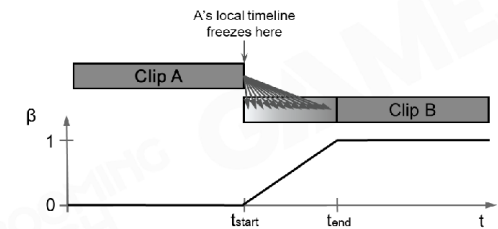
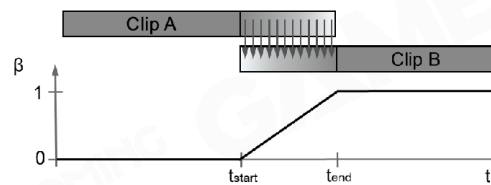


Cross Fades

두 개의 애니메이션 클립을 부드럽게 전환하는 애니메이션 블렌딩 기법

Two common ways

- Smooth transition
 - restriction: the two clips must be looping animations, and their timelines must be synchronized
- Frozen transition



no cross fade

뛰는 동작



smooth transition

부자연스러운 중간 동작



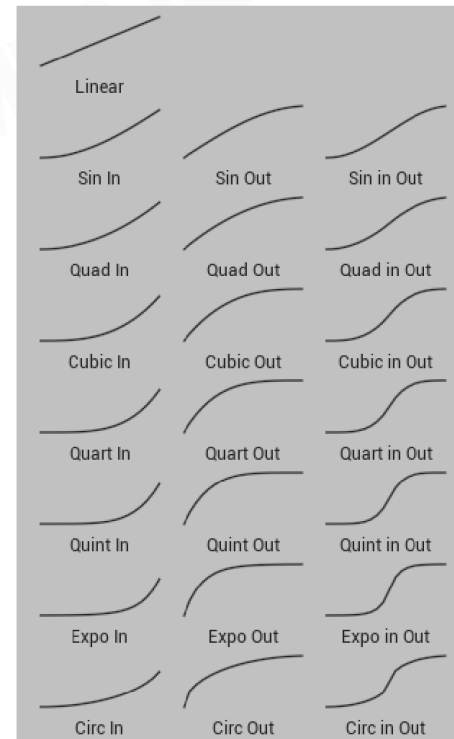
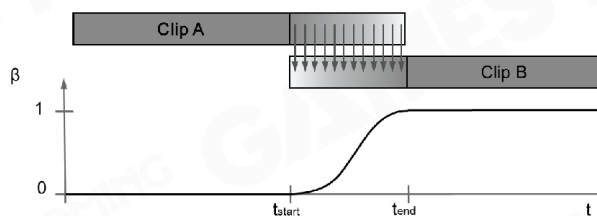
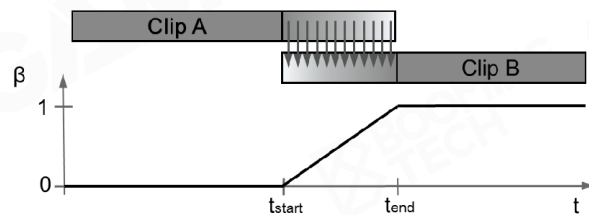
frozen transition

두 동작의 자연스러운 연결



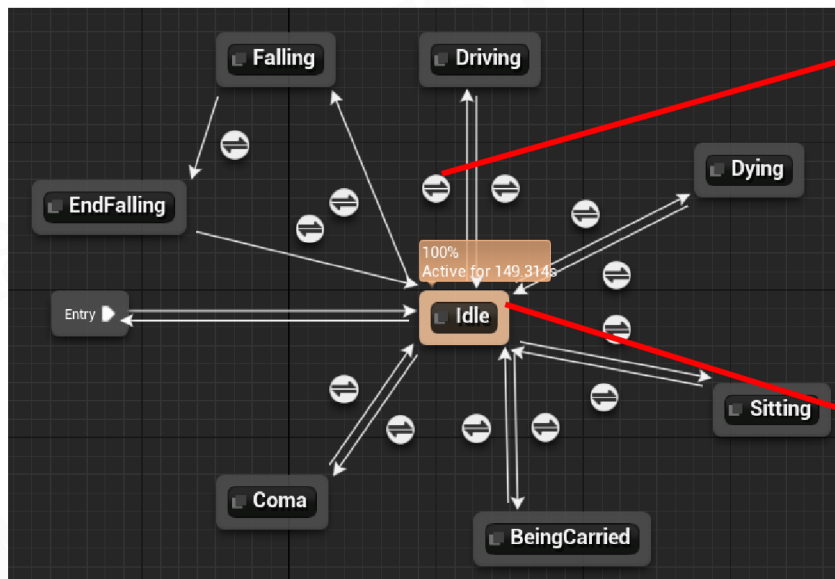
Cross Fades Curve

- Different cross fades curve could be used for different demands

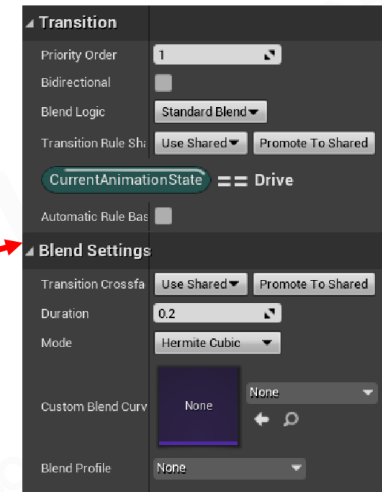


Animation State Machine in Unreal

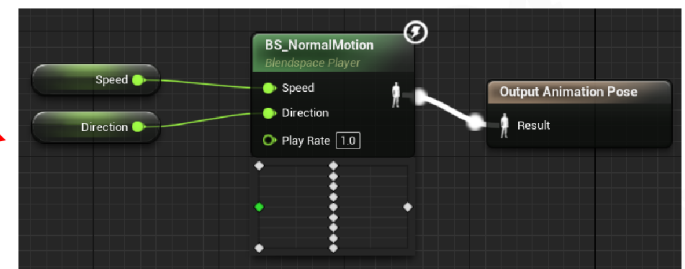
- State: a blueprint graph which outputs a pose
- Transition : control when to change state and how to blend (**Multi**)



Action State Machine



Transition Rule



State Node with Blend Space



Animation Blend Tree



Blend Tree

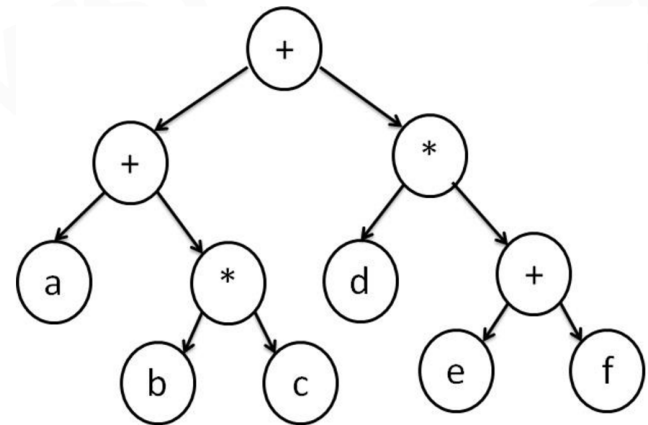
Structure layered ASMs and operations as a tree

- Inspired by expression tree
- Easy to understand for animators

For a blend tree

- Non-terminal nodes and terminal nodes (leaf nodes)
- The result of each non-terminal node is a pose

$$a + (b * c) + d * (e + f)$$

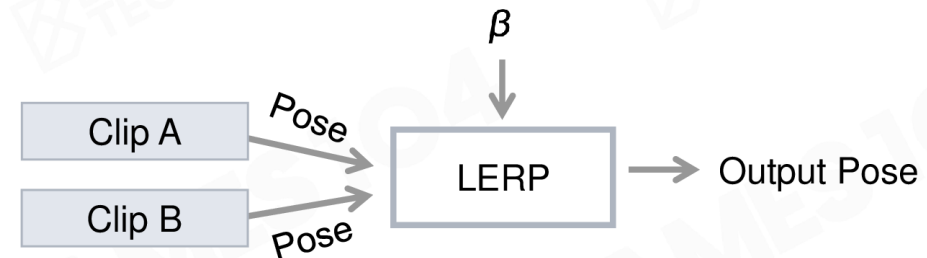


Expression Tree

LERP Blend Node

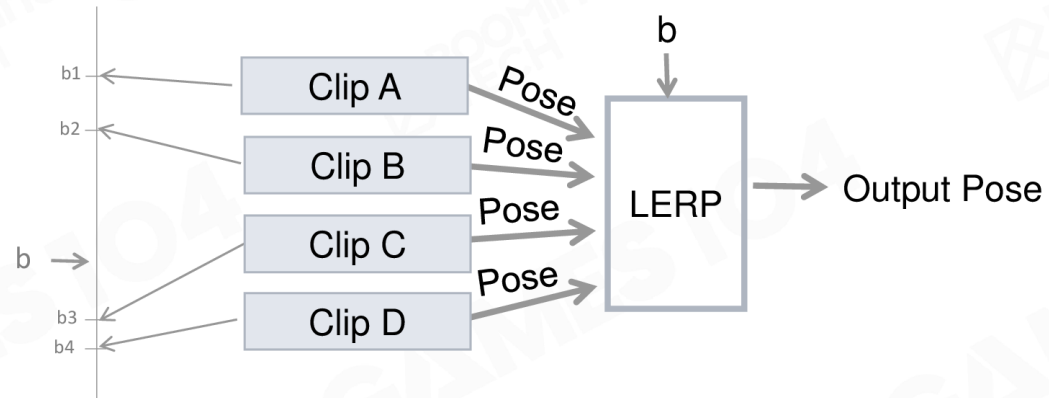
Binary LERP node

- Basic non-terminal node in blend tree
- LERP two input poses with weight β into one output pose



Binary LERP node

Usually extended to handle multiple inputs
(e.g. Ternary/Quad LERP node)



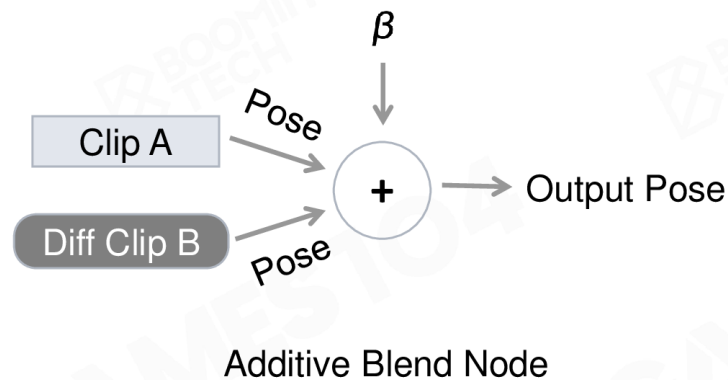
Extended LERP node



Additive Blend Node

Additive Blend node

- Basic non-terminal node in blend tree
- Add the second input pose (usually a difference one) into the first input pose by weight β



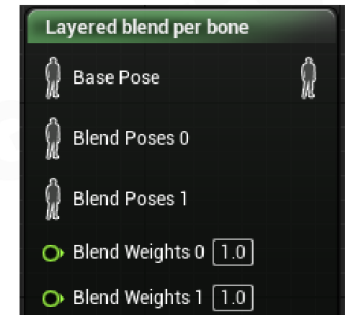
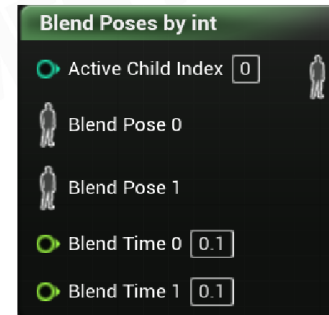
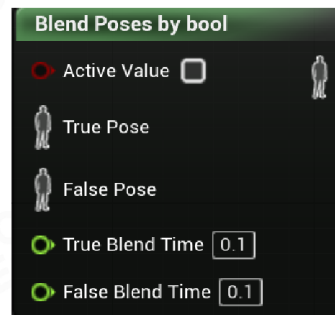
Blend Tree Nodes

Terminal node (Leaf Nodes)

- Clip
- Blend Space
- ASM

Non-terminal node (No-Leaf Nodes)

- Binary LERP blend node
- Ternary (triangular) LERP blend node
- Binary additive blend node

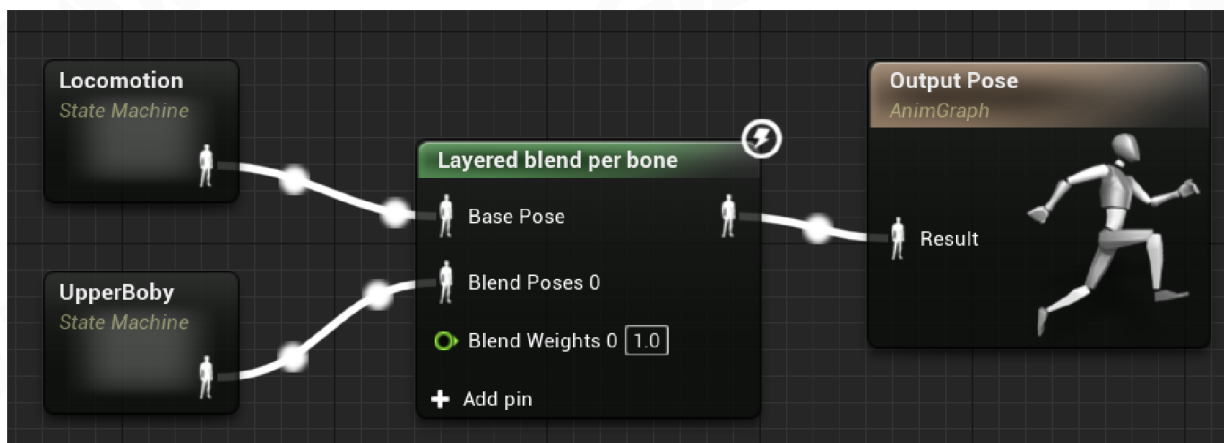


Nodes in UE4

Unreal Animation Blueprint

A blueprint graph which outputs a final pose

- Take clip poses or the results of ASMs as input
- Blend input poses by different methods





Inverse Kinematics (IK)



Basic Concepts

- **End-effector**

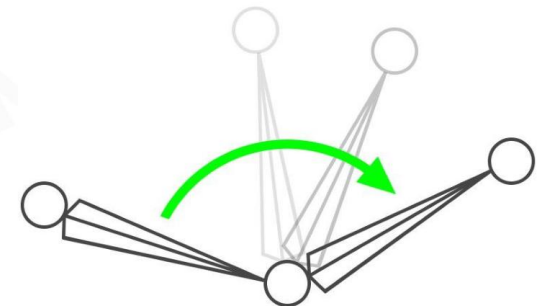
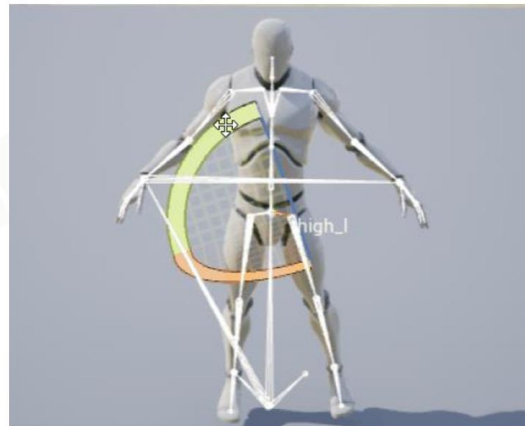
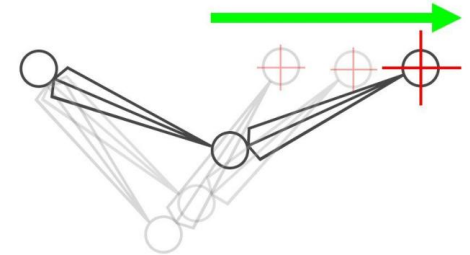
The bone which is expected to be moved to a desired position

- **IK (Inverse Kinematics)**

The use of kinematic equations to **determine the joint parameters** of a manipulator so that the end-effector moves to a desired position

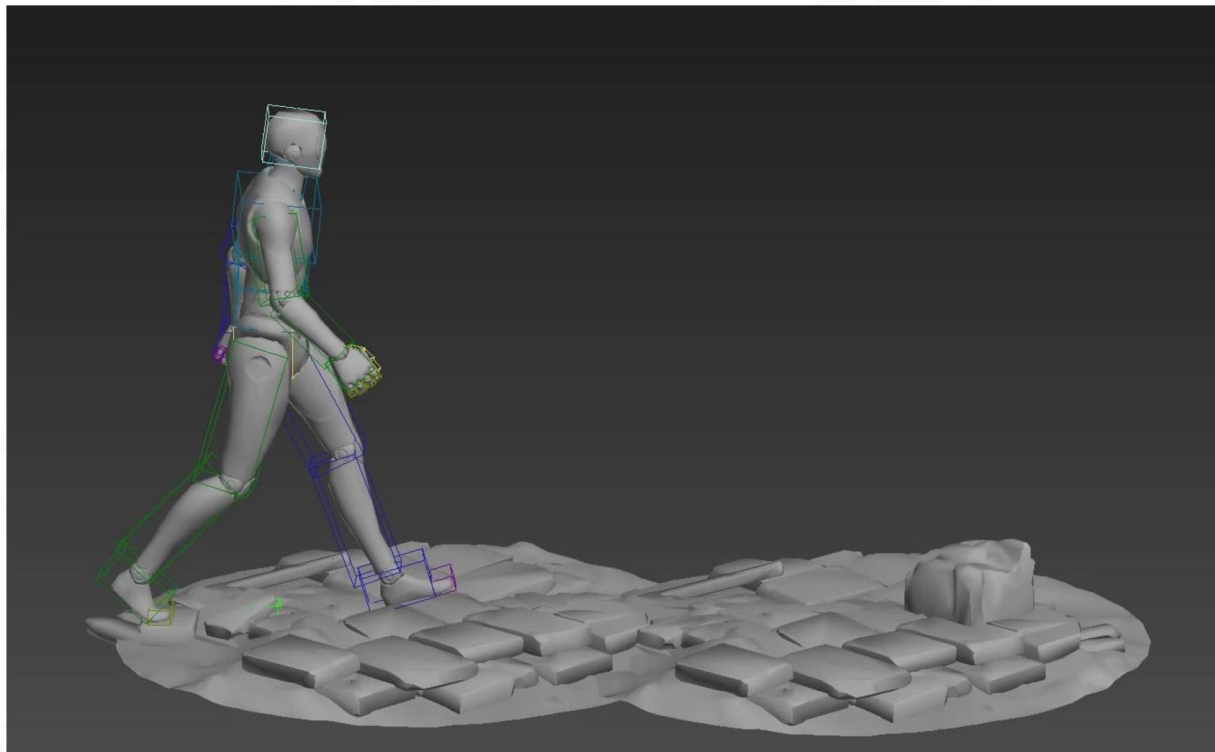
- **FK (Forward Kinematics)**

The use of the kinematics equations of a robot to **compute the position of the end-effectors** from specified values for the joint parameters



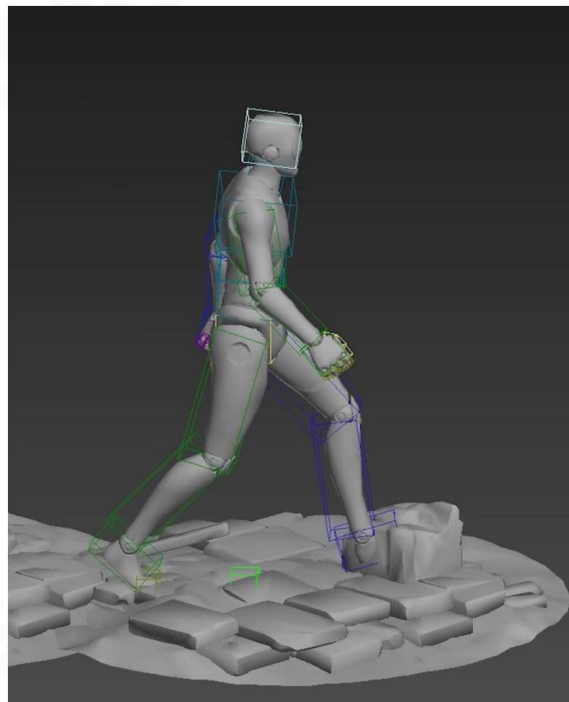


How to Touch the Ground?



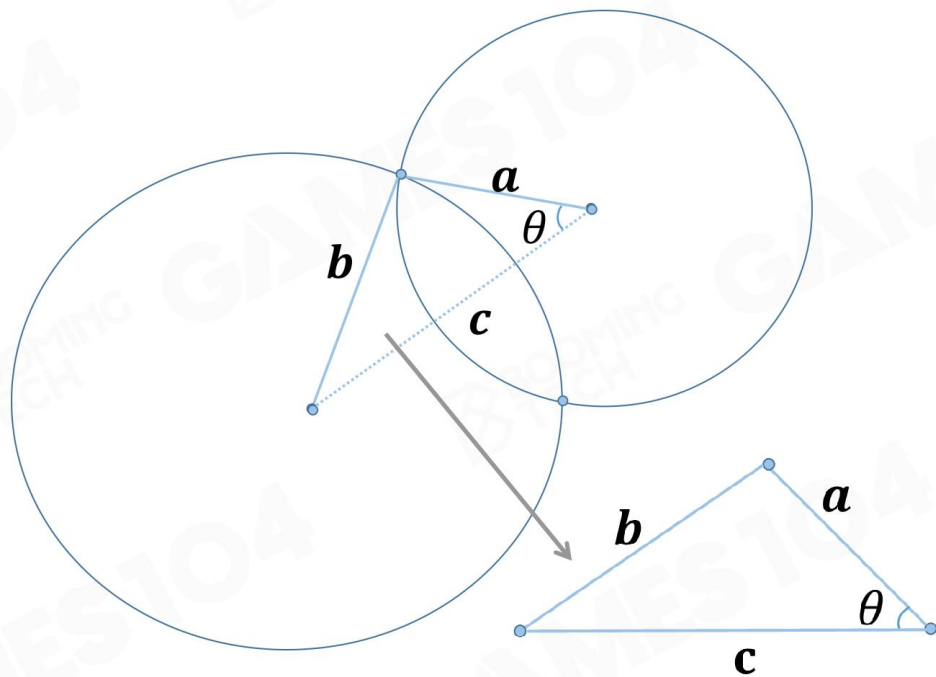
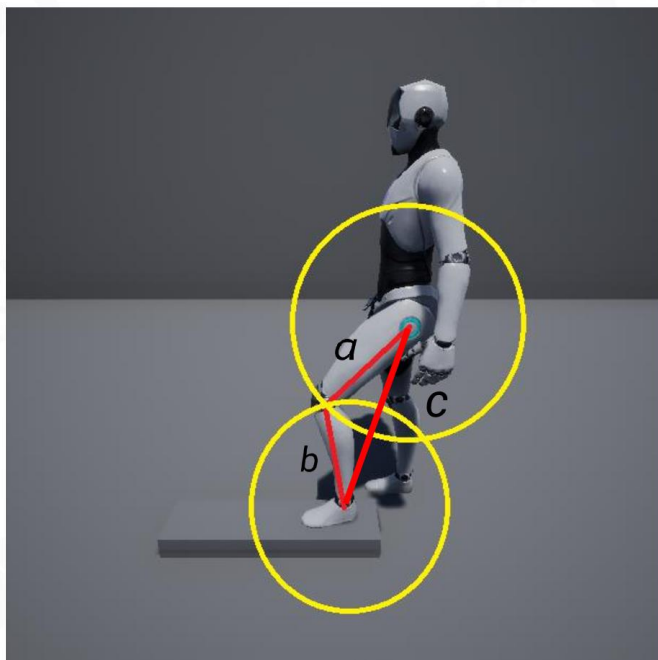


Intuitive Idea: Adjust Feet Position for Each Step





Two Bones IK

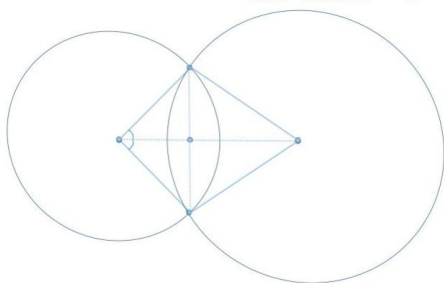


$$\cos(\theta) = \frac{a^2 + c^2 - b^2}{2ac}$$

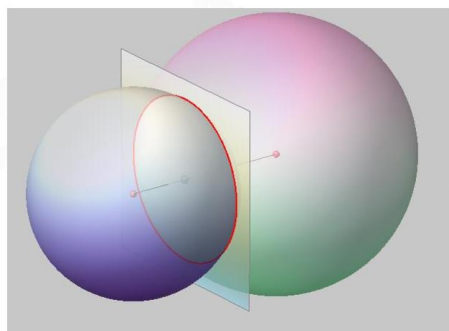


Two Bones IK

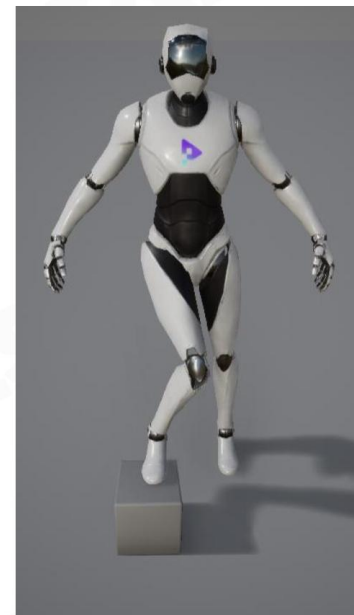
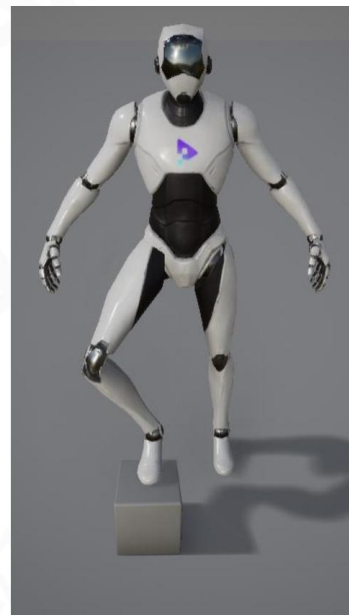
- 3D space



2D



3D

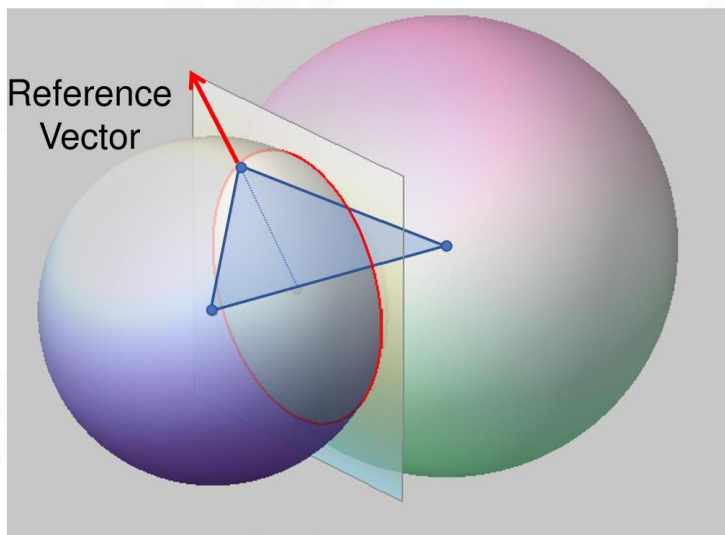




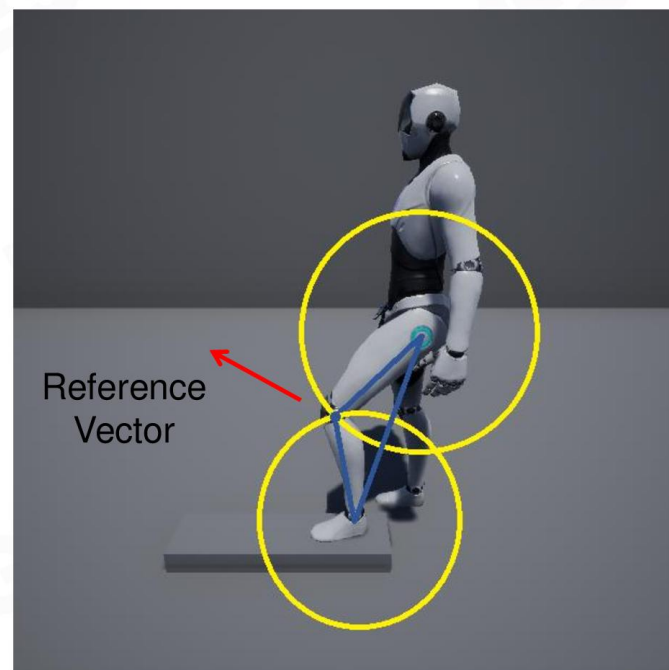
Two Bones IK

- 3D space
- Determine the final pose by a reference vector

IK 풀이를 할 때 목표 지점(Target)에 도달하기 위해 사용하는 "기준 방향 벡터"



3D View



2D View of the Final Plane



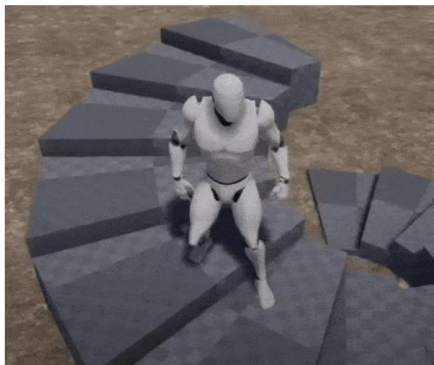
More Complicated IK Scenarios



Look At IK



Hand IK



Foot IK



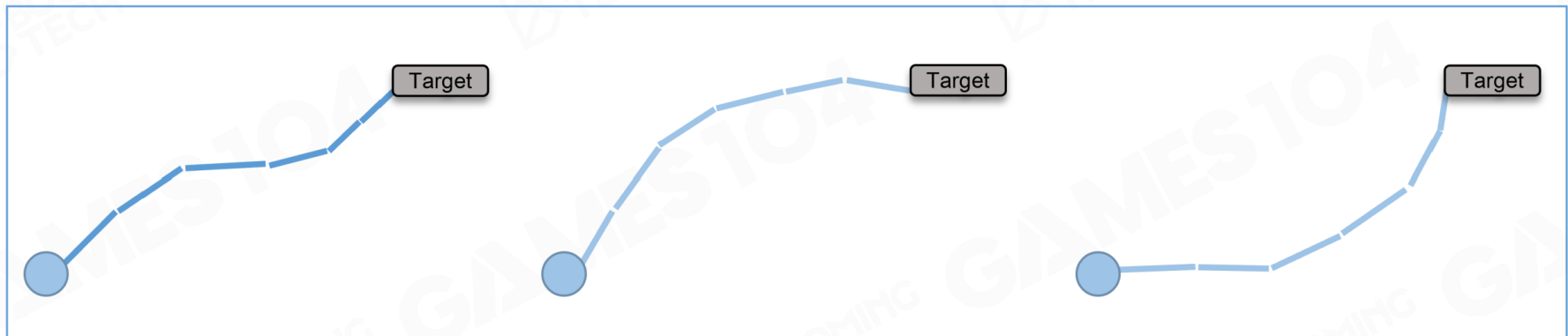
Full Body IK



Complexity of Multi-Joint IK Solving

- Computation cost: high dimension non-linear function solving in real-time
- May have multiple solutions, unique solution or no solution

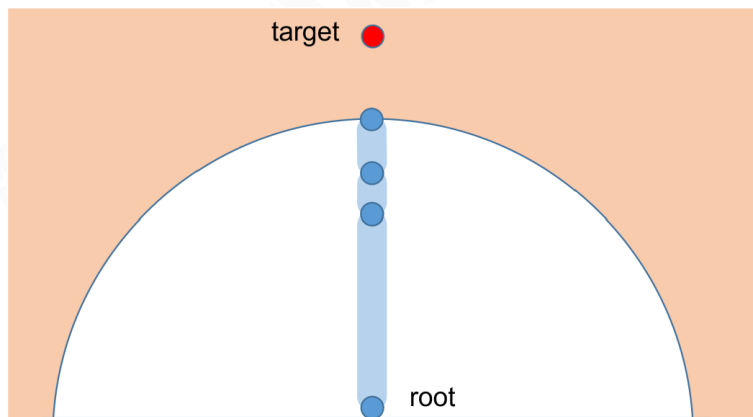
Target



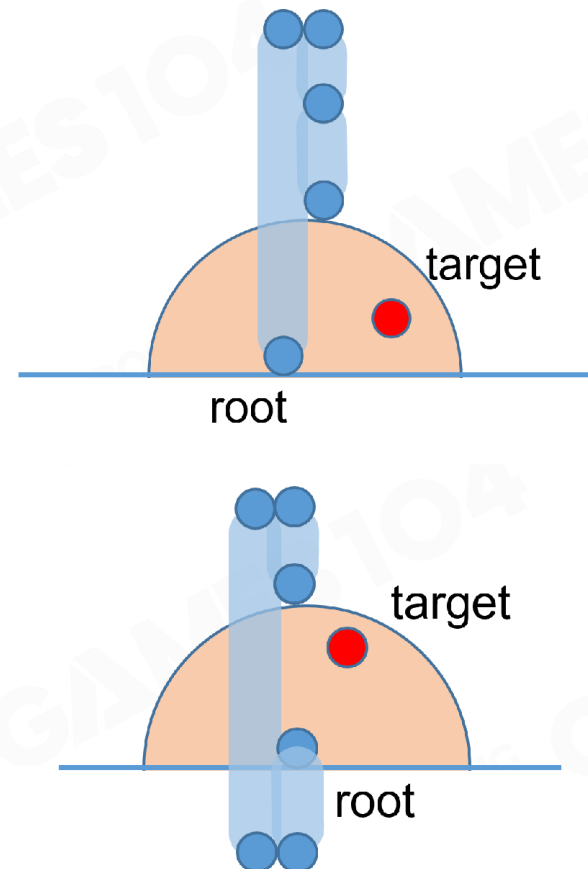
Multiple Solution



Check Reachability of the Target

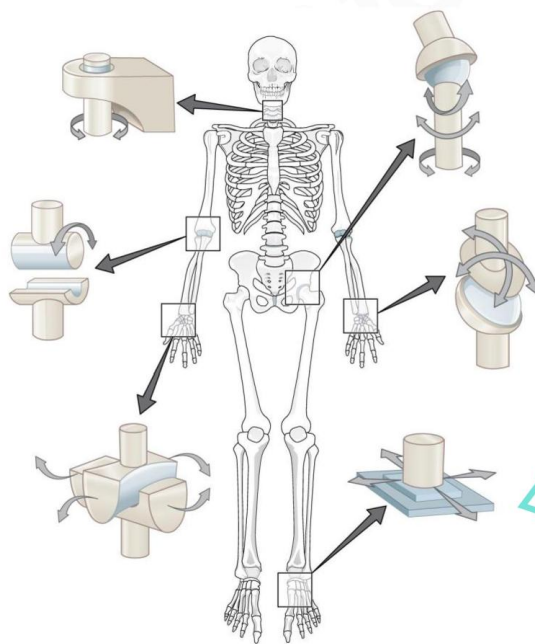


or





Constraints of Joints



Constraints of Human Skeleton



Hinge



Ball-and-socket



Pivot



Saddle



Condylod



Gliding



Heuristics Algorithm

Why

- Too many joints + constraints, difficult to solve with analysis method

Basic Idea

Designed to solve problem in faster and more efficient fashion by sacrificing optimality, accuracy, precision, or completeness for speed

- Approximation
- Global optimality is not guaranteed
- Iteration is usually used with a maximum limit



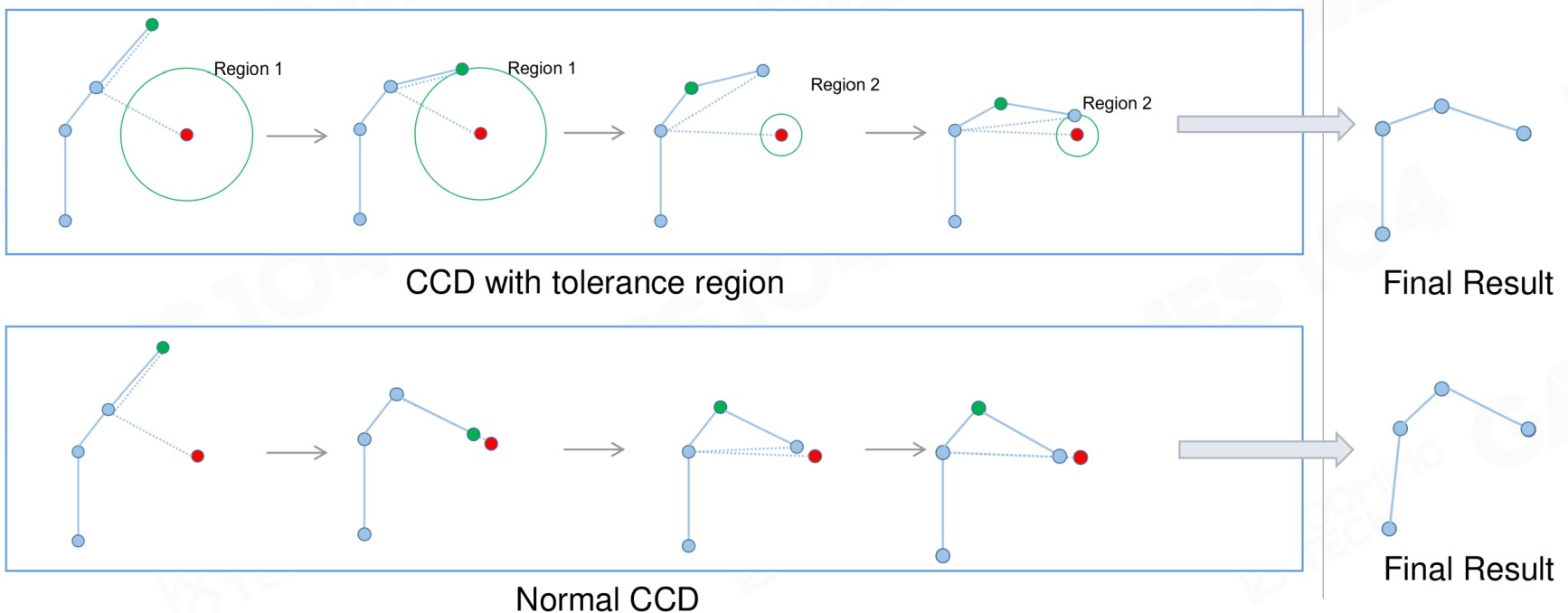


Optimized CCD

Cyclic Coordinate Descent: 한 번에 하나의 관절만 조정하며, 끝에서부터 시작해서 목표에 가깝게 만든다.

Add tolerance regions to each bone's goal

- Each bone stops rotating and moves onto the next bone within tolerance region
- Helps to produce poses that are less rigid and more comfortable looking





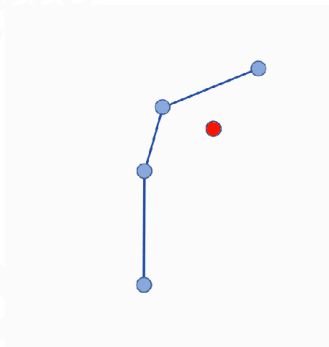
FABRIK (Forward And Backward Reaching Inverse Kinematics)

Principle

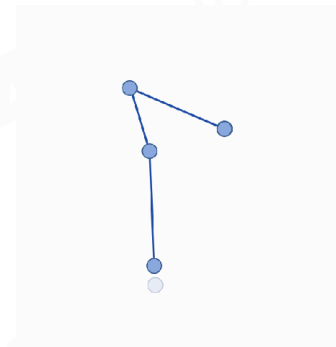
- Instead of orientation space, solves IK problem in position space

Reachability

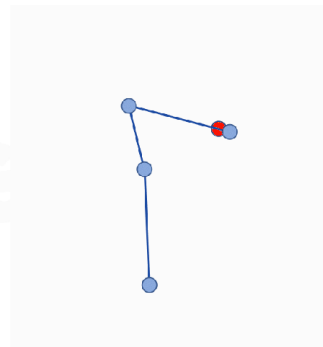
- Algorithm can stop after certain number of iterations to avoid unreachable target problem



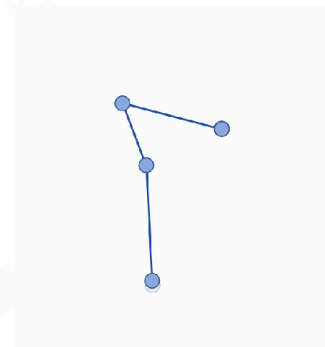
Forward(1)



Backward(2)



Forward(3)



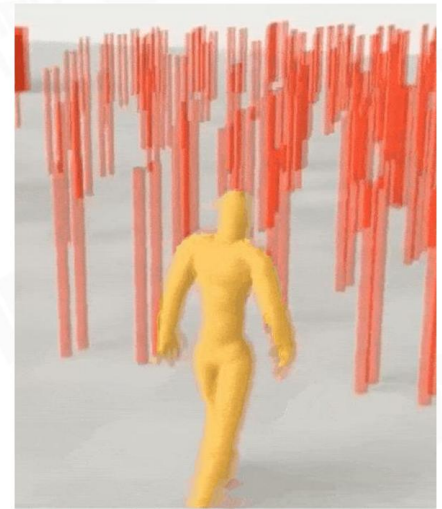
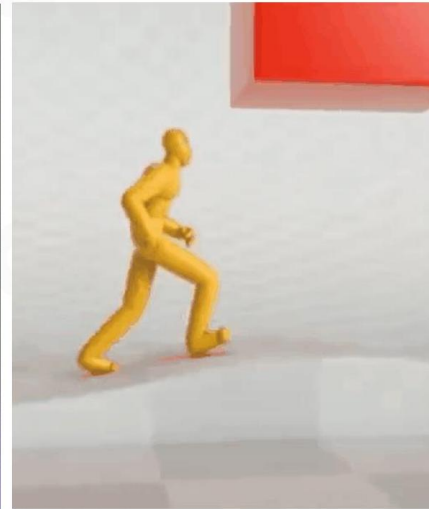
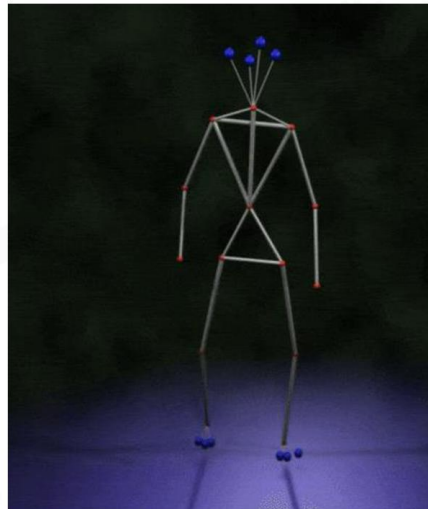
Backward(4)

- 위치 기반(position-based)으로 반복적(iterative)으로 해결하는 현대적이고 효율적인 알고리즘
- CCD와 달리 회전을 직접 계산하지 않고, 관절 위치를 앞뒤로 조정해 목표에 도달



Multiple End-Effectors

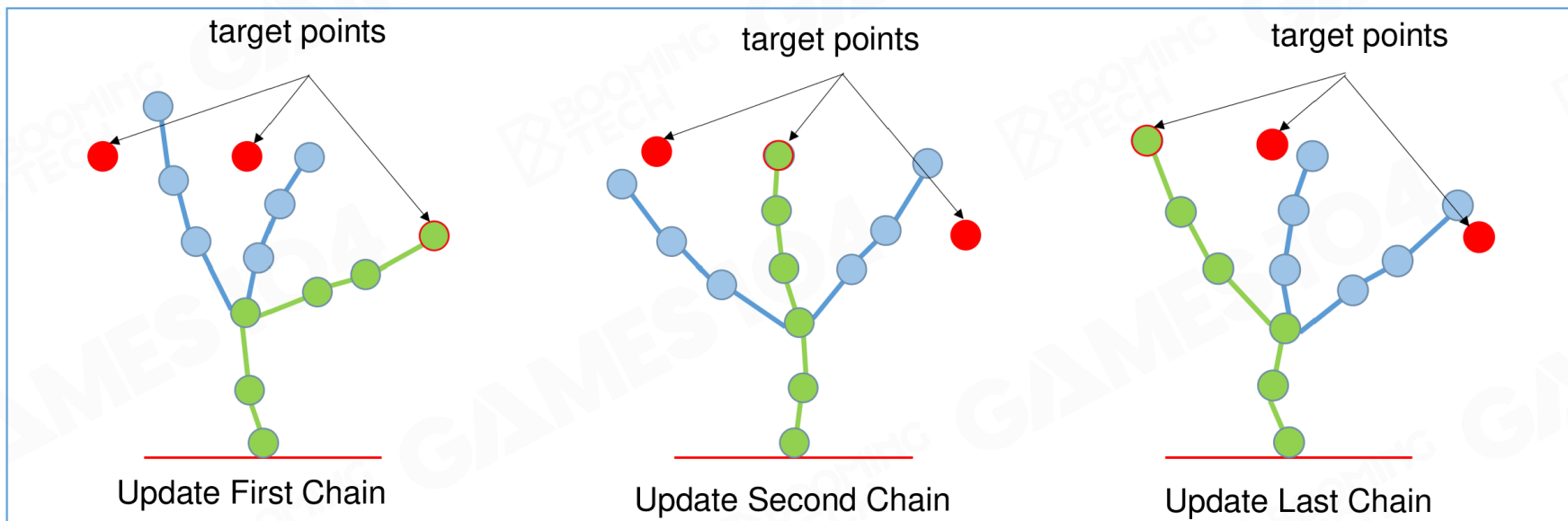
- May result in conflict between goals, which can not be achieved simultaneously
- May use a priority or a weighted approach





IK with Multiple End-Effectors

- If a shared bone needs to be moved, the end-effector that is updated last will get priority and the other bones will be pulled away



Jacobian Matrix

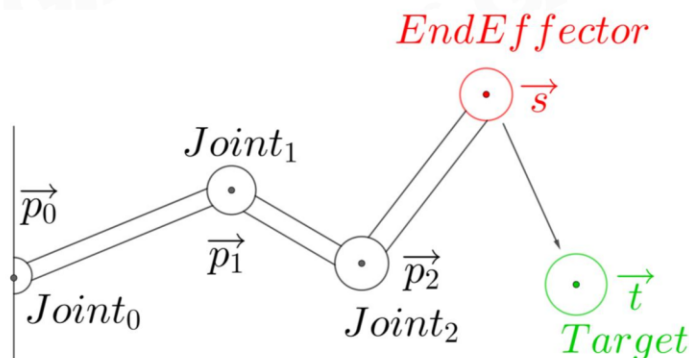
In vector calculus, the Jacobian Matrix of a vector-valued function of several variables is the matrix of all its first-order partial derivatives

Suppose:

$$\vec{f}(\vec{x}) = \begin{bmatrix} f_1(\vec{x}) \\ f_2(\vec{x}) \\ \vdots \\ f_m(\vec{x}) \end{bmatrix} \quad \vec{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

then, the Jacobian Matrix of $\vec{f}(\vec{x})$:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$$



핵심: “목표 위치에 대한 각 관절 각도의 변화율” → 미분(derivative) → 선형 근사 → 반복적 갱신

기호	의미
$\mathbf{q}$	관절 각도 벡터 (예: $[q_1, q_2, q_3]$)
$\mathbf{p}$	끝효과기 위치 (손/발 위치)
$\mathbf{f}(\mathbf{q})$	순기구학 (forward kinematics)
$\mathbf{J}(\mathbf{q})$	Jacobian 행렬 = $\frac{\partial \mathbf{p}}{\partial \mathbf{q}}$

$$\Delta \mathbf{p} = \mathbf{J}(\mathbf{q}) \cdot \Delta \mathbf{q}$$



Jacobian Matrix with Multiple End-effectors

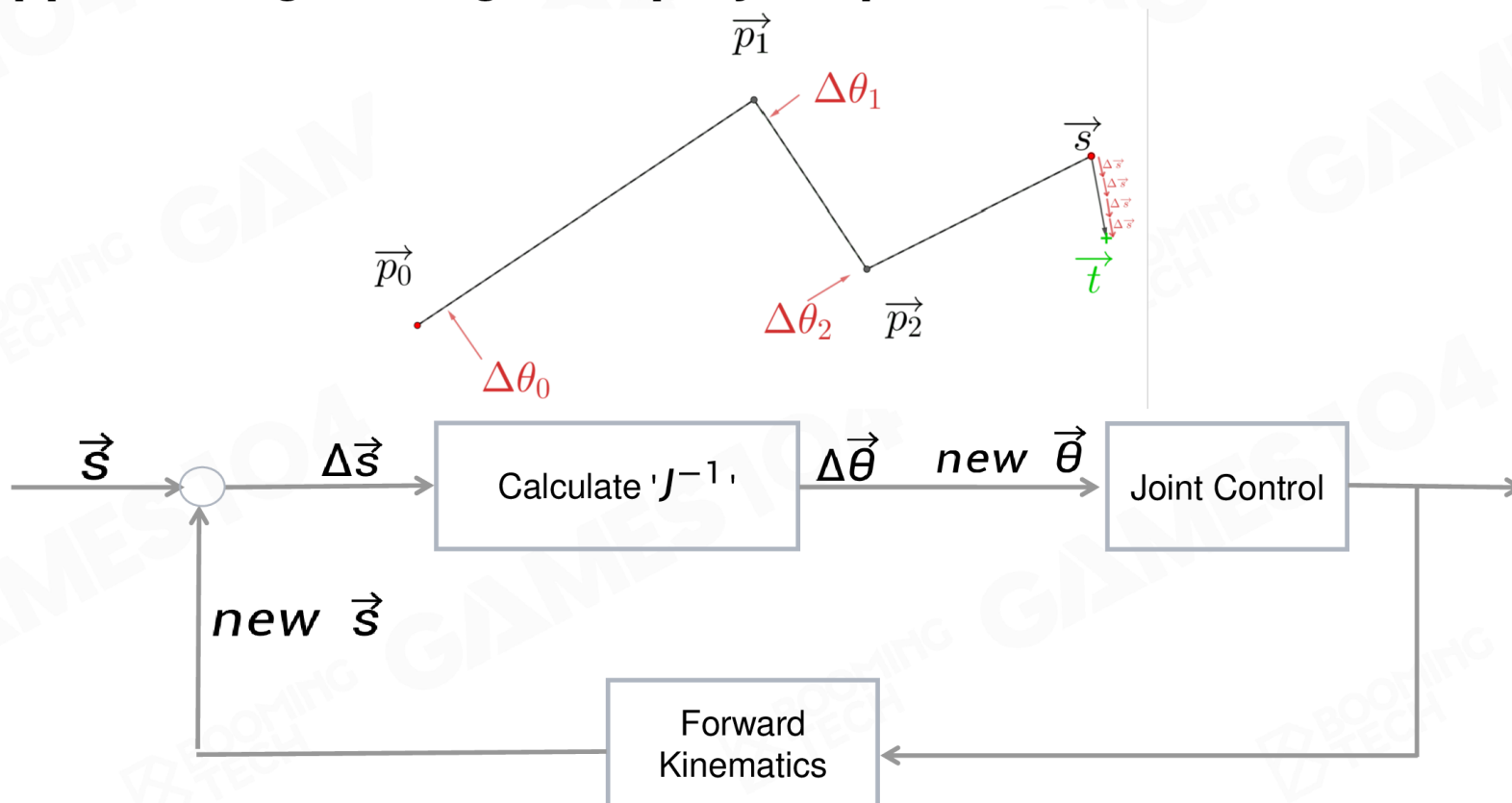
$$J = \begin{bmatrix} \frac{\partial \vec{s}_1}{\partial \theta_1} & \frac{\partial \vec{s}_1}{\partial \theta_2} & \cdots & \frac{\partial \vec{s}_1}{\partial \theta_n} \\ \frac{\partial \vec{s}_2}{\partial \theta_1} & \frac{\partial \vec{s}_2}{\partial \theta_2} & \cdots & \frac{\partial \vec{s}_2}{\partial \theta_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \vec{s}_m}{\partial \theta_1} & \frac{\partial \vec{s}_m}{\partial \theta_2} & \cdots & \frac{\partial \vec{s}_m}{\partial \theta_n} \end{bmatrix}$$

m : the number of end-effectors

n : the number of joints



Approaching to Target Step by Step





Other IK Solutions

Physics-based Method

- More natural
- Usually need lots of computation if no optimization

PBD(Position Based Dynamics)

- Different from traditional physics-based method
- Better visual performance
- Lower computational cost

Fullbody IK in UE5

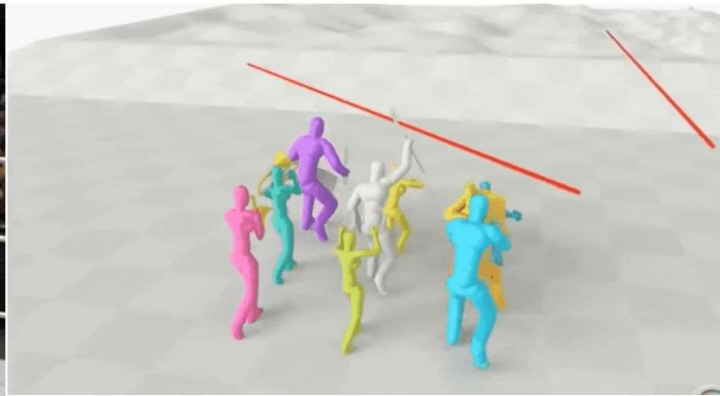
- XPBD (Extended PBD)



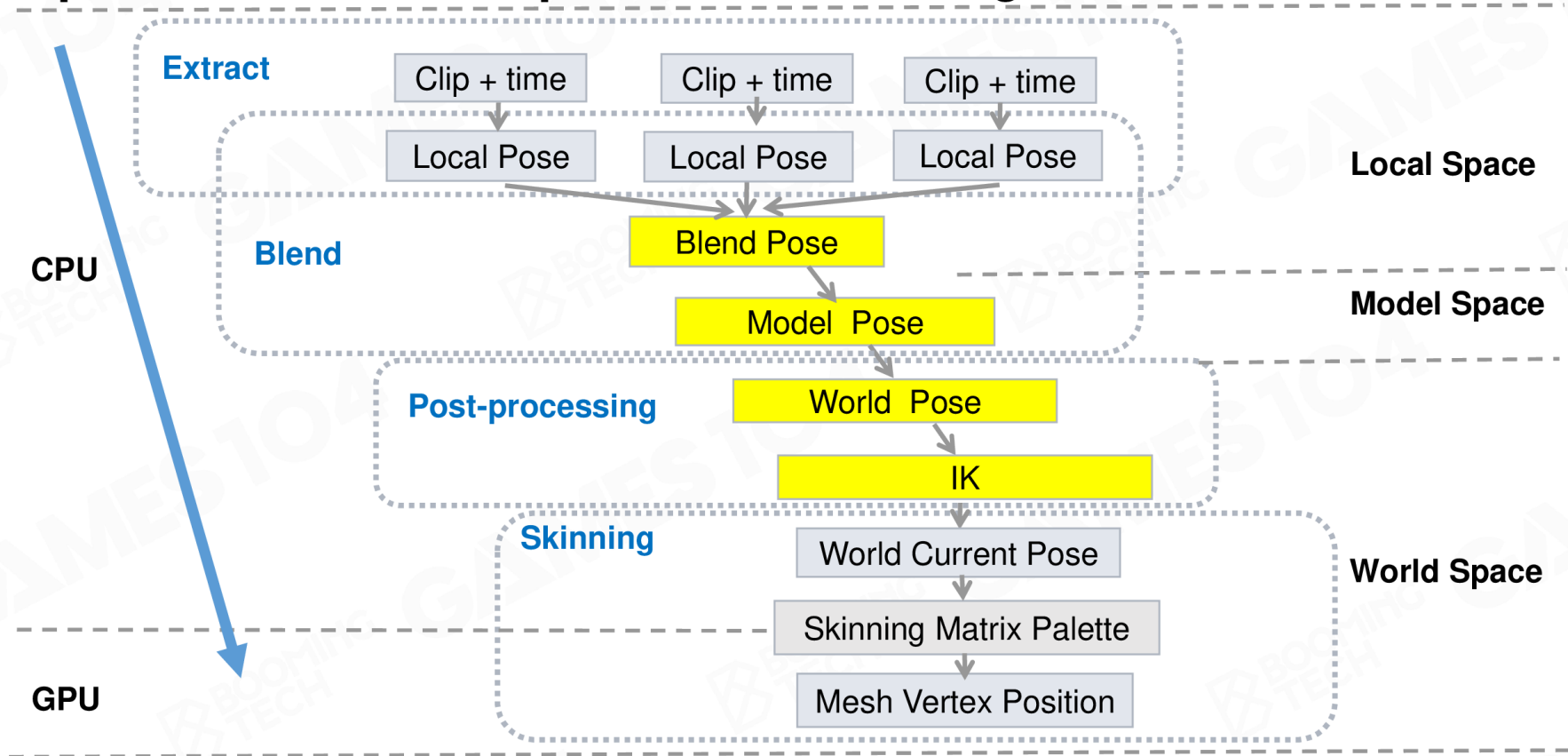


IK is still Challenge

- Self collision avoidance
- IK with predication during moving
- Natural human behavior
 - Data-driven and deep learning



Updated Animation Pipeline with Blending and IK



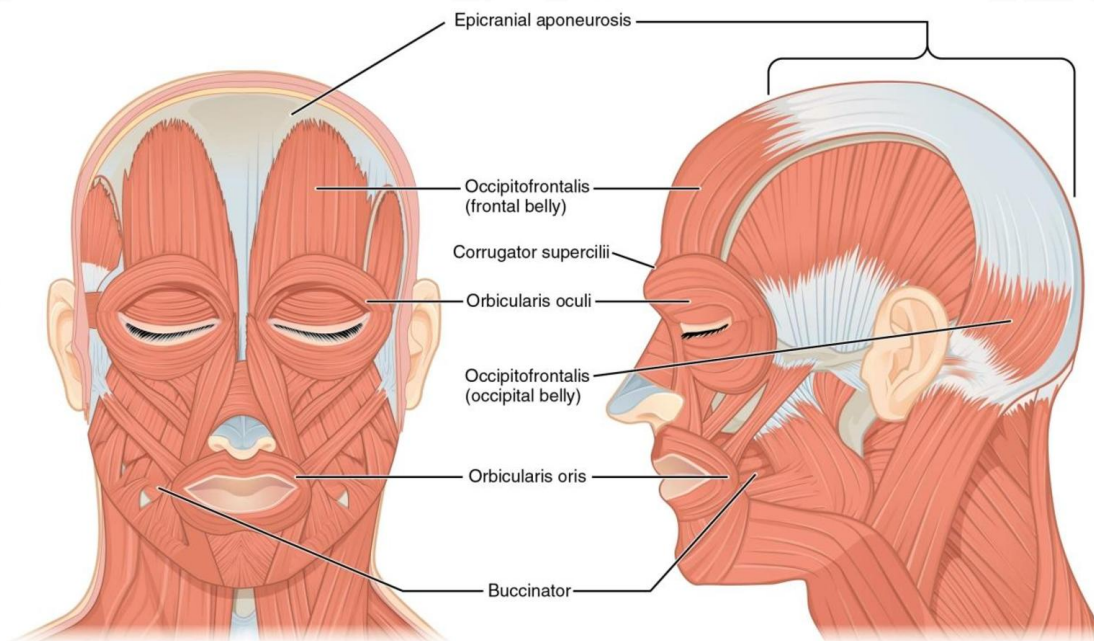


Facial Animation





Face is Driven by Complex Muscle System



Facial Muscles

- 43 Muscles
- Variant shape, strength and movement
- Work together to make expressions



Facial Action Coding System

Facial Action Coding System (FACS) is a system to taxonomize human facial movements by their appearance on the face.



Upper Face Action Units					
AU 1	AU 2	AU 4	AU 5	AU 6	AU 7
Inner Brow Raiser	Outer Brow Raiser	Brow Lowerer	Upper Lid Raiser	Cheek Raiser	Lid Tightener
*AU 41	*AU 42	*AU 43	AU 44	AU 45	AU 46
Lid Droop	Slit	Eyes Closed	Squint	Blink	Wink
Lower Face Action Units					
AU 9	AU 10	AU 11	AU 12	AU 13	AU 14
Nose Wrinkler	Upper Lip Raiser	Nasolabial Deepener	Lip Corner Puller	Cheek Puffer	Dimpler
AU 15	AU 16	AU 17	AU 18	AU 20	AU 22
Lip Corner Depressor	Lower Lip Depressor	Chin Raiser	Lip Puckerer	Lip Stretcher	Lip Funneler
AU 23	AU 24	*AU 25	*AU 26	*AU 27	AU 28
Lip Tightener	Lip Pressor	Lips Part	Jaw Drop	Mouth Stretch	Lip Suck

Part of 46 basic movements are named action units(AU)



Action Units Combination

An expression can be considered as a combination of some of the basic movements



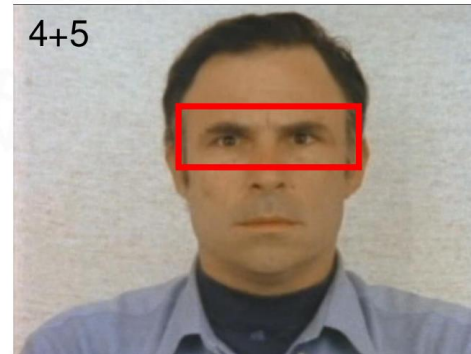
Brow Lowerer

+



Upper Lid Raiser

=



'4+5' Combination



28 Core Action Units

- Apple Inc. extracted the 28 core AUs
- 23 Symmetric AUs are divided into two basic actions
- Basic actions set varies according to the animation production requirements

Neutral face	Lip corner depressor
Inner brow raiser	Lower lip depressor
Outer brow raiser	Chin raiser
Brow lowerer	Lip pucker
Upper lid raiser	Tongue show
Cheek raiser	Lip stretcher
Lid tightener	Neck tightener
Lips toward each other	Lip funneler
Nose wrinker	Lip tightener
Upper lip raiser	Lip pressor
Nasolabial deepener	Lips part
Lip corner puller	Jaw drop
Sharp lip puller	Mouth stretch
Dimpler	Lip suck

28 Core AUs



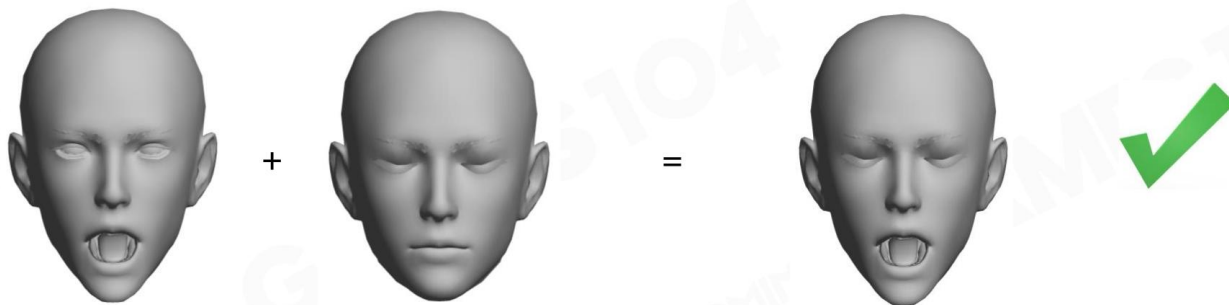
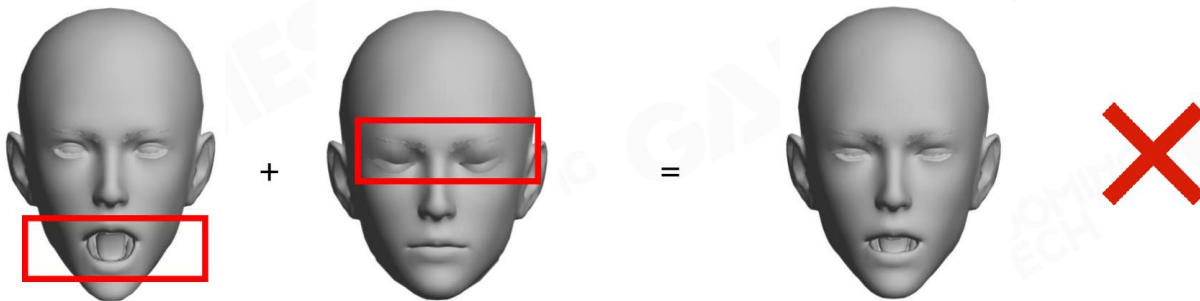
Key Pose Blending

A set of key poses (a variation on per-vertex animation)





Problems of Simple Blending



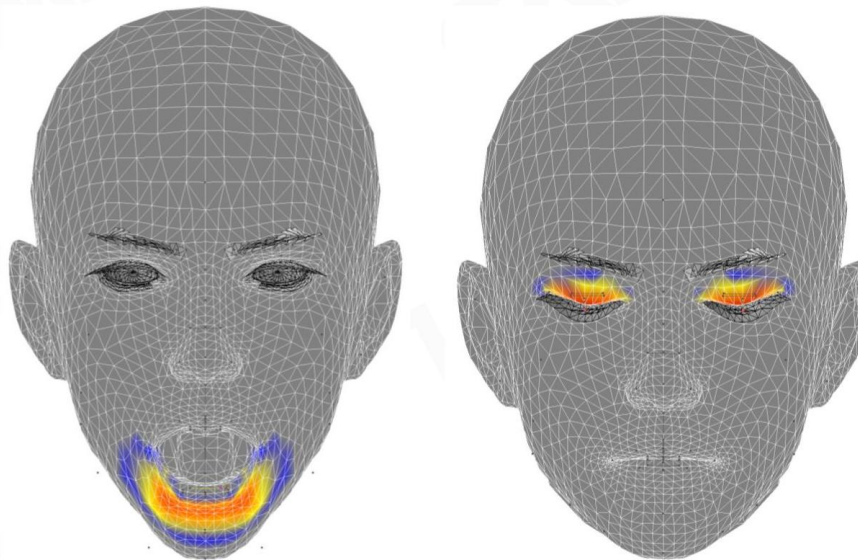
Mouse Open

Eye Close



FACS In Morph Target Animation

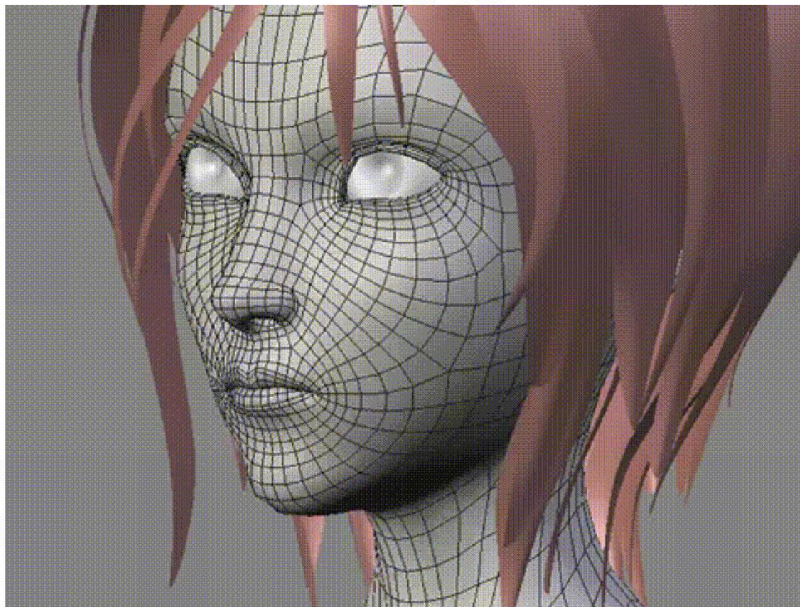
- Create AU key poses only store vertexes different from the neutral pose (Additive Blending)



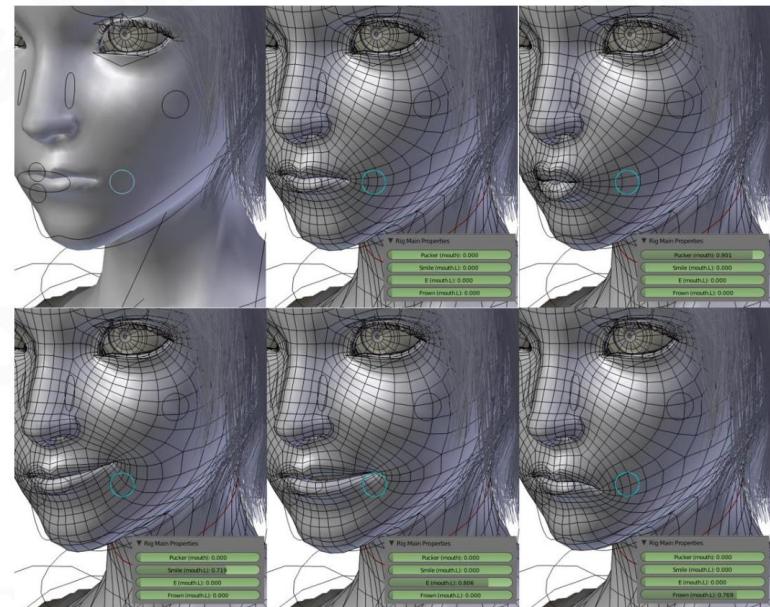
Vertex offset from neutral face



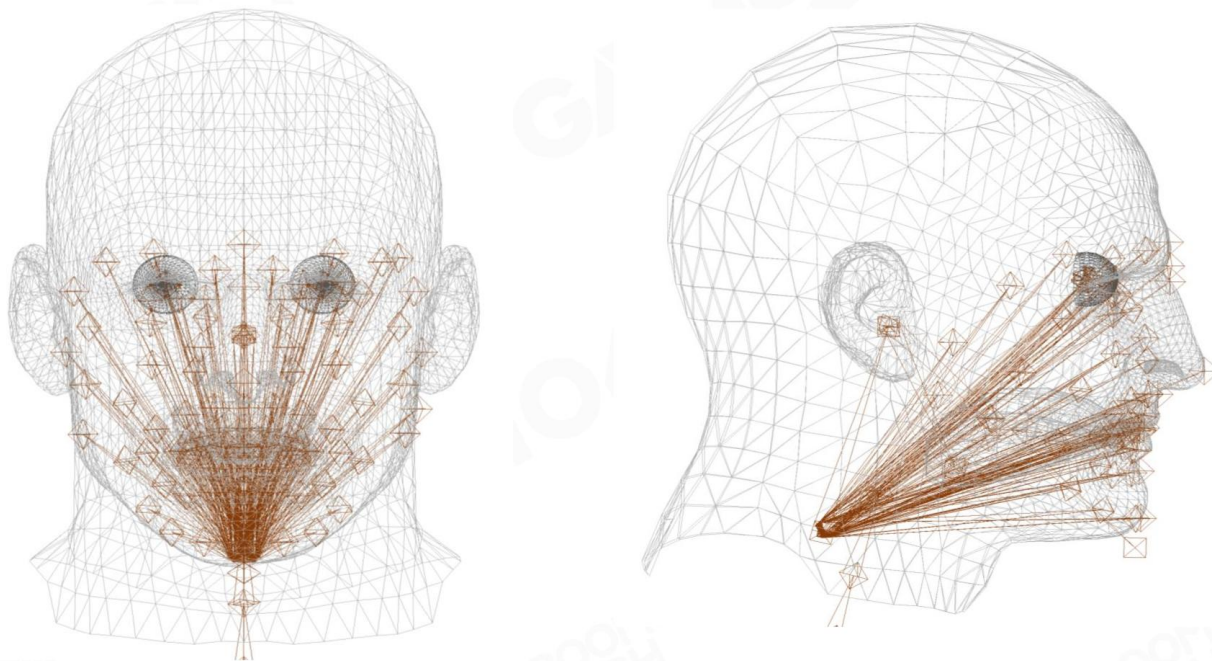
Morph Target Animation



Facial animation by morphing among key poses



Key Poses



Complex Facial Skeleton



UV Texture Facial Animation

- Using a series of texture maps applied to a simple head shape



Animal Crossing: New Horizons



The Legend of Zelda: Breath of the Wild

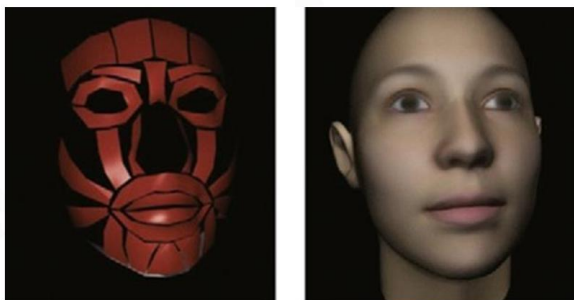


Muscle Model Animation

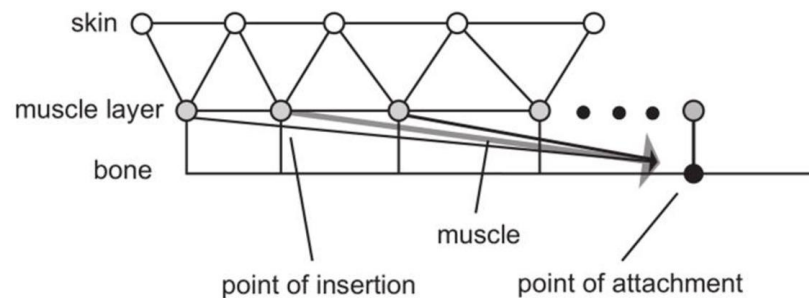
In the reliance on a physical basis, more precise, but more sophisticated

- Muscle controls most part of the face
- 3 Layers: Skin Layer, Muscle Layer, Bone Layer
- The point of insertion will move an amount determined by the muscle

The model used for the skin will dictate how the area around the insertion point muscle reacts



Muscle Spreads Over The Face



Cross Section of The Muscle Model



Animation Retargeting

한 캐릭터의 애니메이션을 다른 캐릭터에게 자연스럽게 적용하는 기술

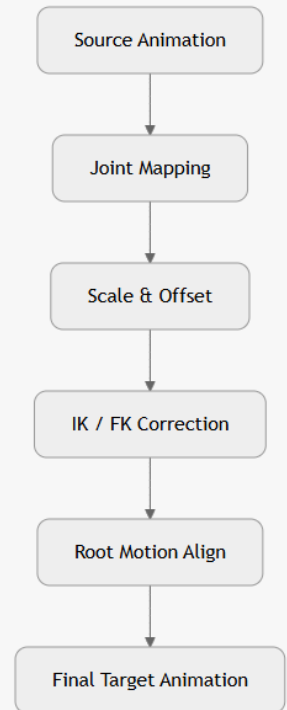
상황	문제	해결책
성인 → 어린이	팔 길이 다름 → 손이 땅 뚫림	리타게팅
인간 → 몬스터	다리 4개 → 2개	매핑 + IK 보정
모션 캡처 → 게임 캐릭터	배우 180cm → 캐릭터 160cm	스케일링 + 보정



Share Animation Among Characters



- Allow animations to be reused between characters (save animator's work)
- Adapt motion captured animations to different characters (reduce the cost)





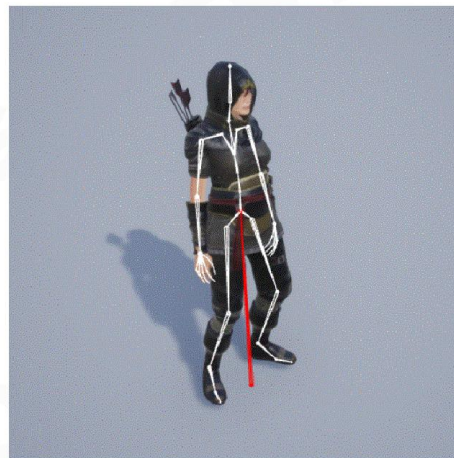
Terminology



Source Character



Target Character



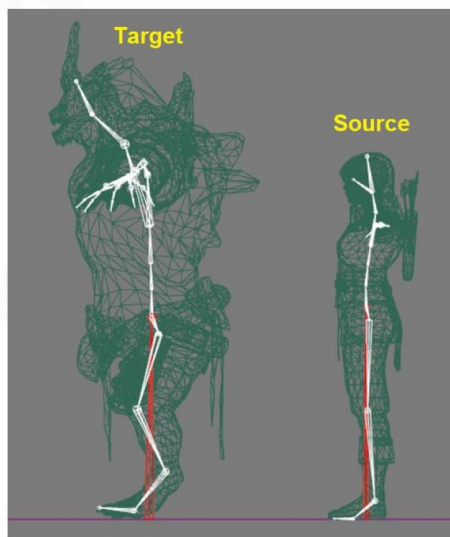
Source Animation



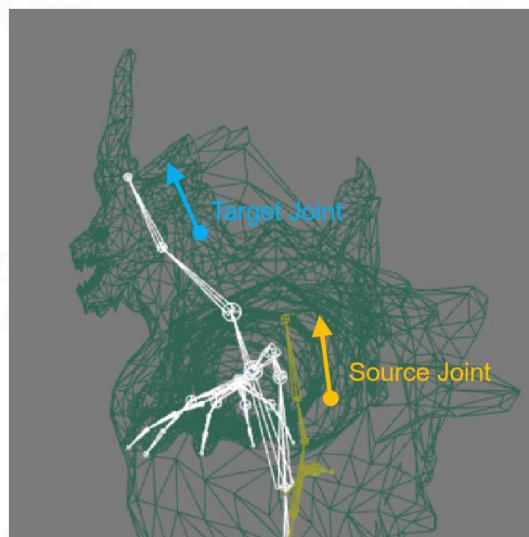
Target Animation



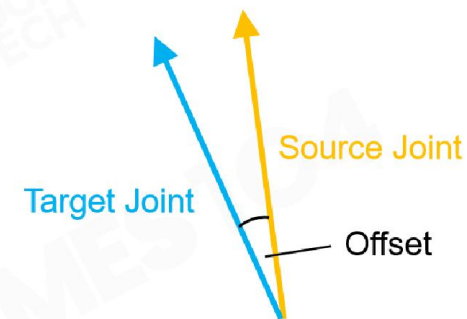
Ignore Offset Between Source and Target Joints



Source vs. Target at Retarget Pose



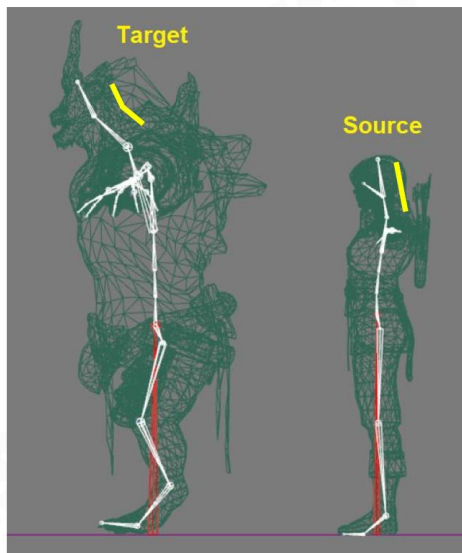
(Source skeleton in yellow)



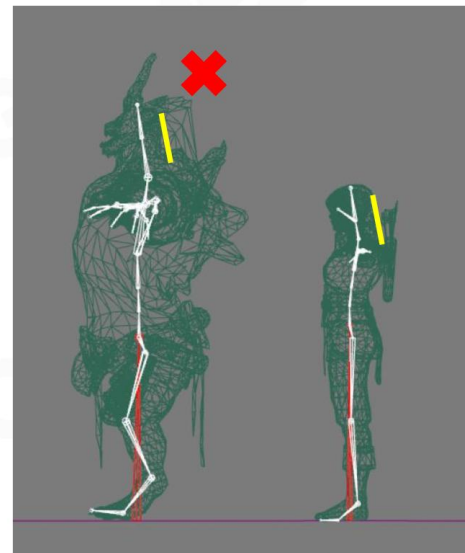
The Offset in Retarget Pose



Keep Orientation in Different Binding Pose



Target vs. Source at retarget pose



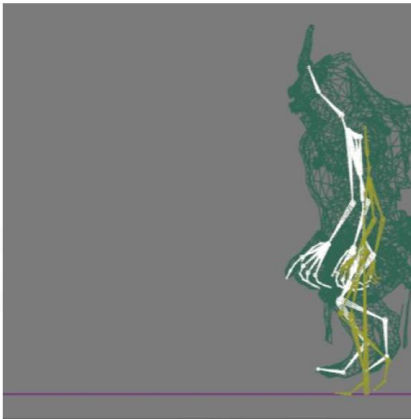
The target looks weird



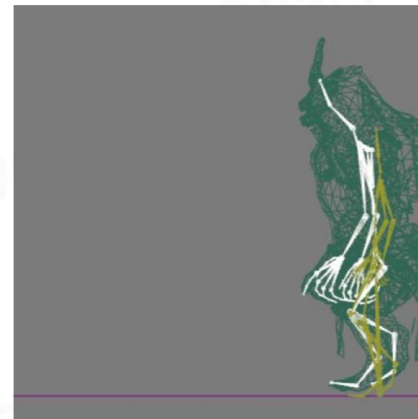
Align Movement by Pelvis Height

The movement of the character

- Usually controlled by displacement curve or motor system at runtime
 - Displacement Curve is extracted from the pelvis pose in animation
- Needs to be scaled by the proportion of the pelvis

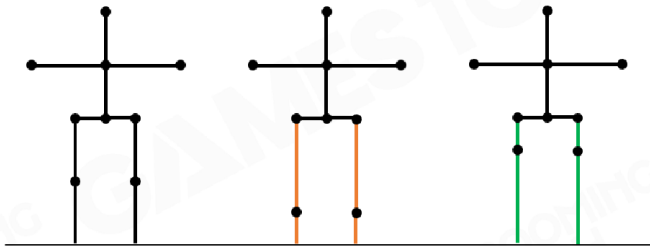


Hanging feet without movement scale
(Source animation in yellow)

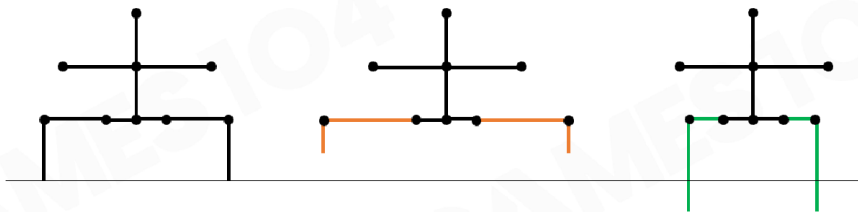


Problem eased with movement scale
(Source animation in yellow)

Lock Feet by IK after Retargeting



Source Skeleton(left) vs. Target Skeleton with longer thigh(middle) or longer calf(right)



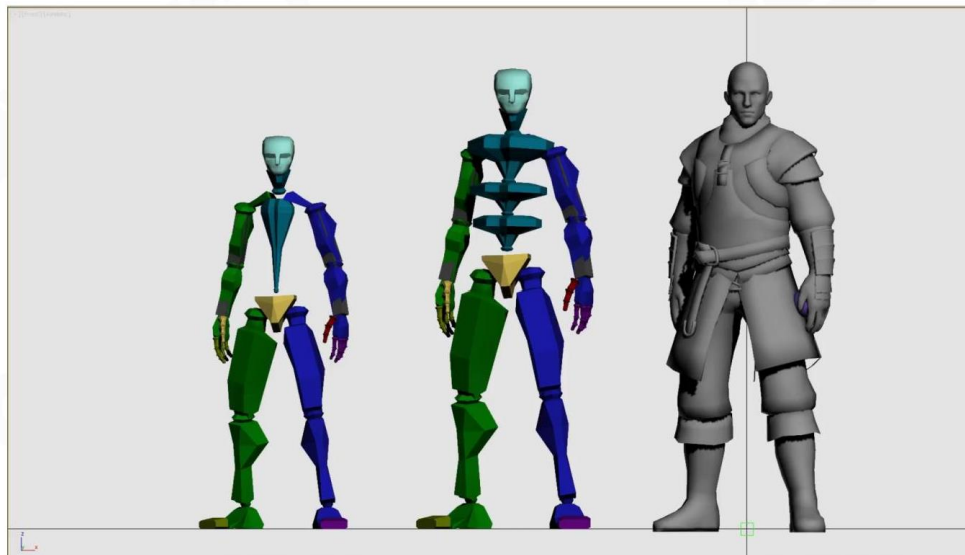
If the thigh is horizontal (left), longer thigh results in hanging feet (middle) while longer calf results in penetration (right)



Target animation with foot IK



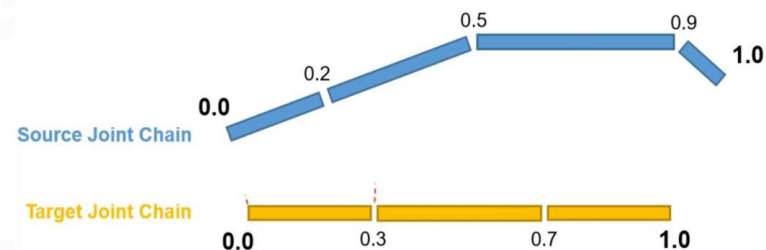
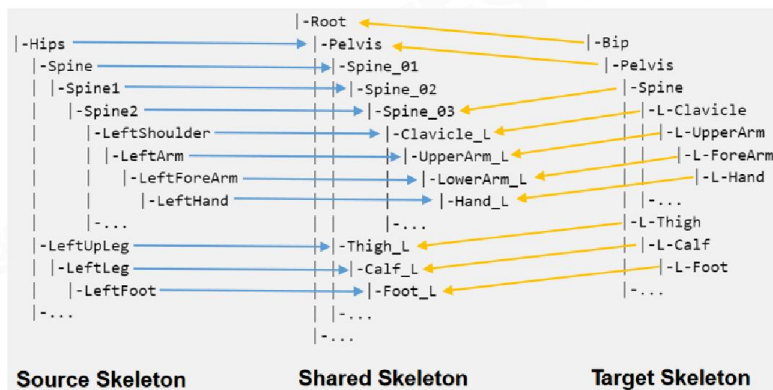
Retargeting with Different Skeleton Hierarchy



Source Skeleton with 1 spine (left) vs.
Target Skeleton with 3 spines (center)



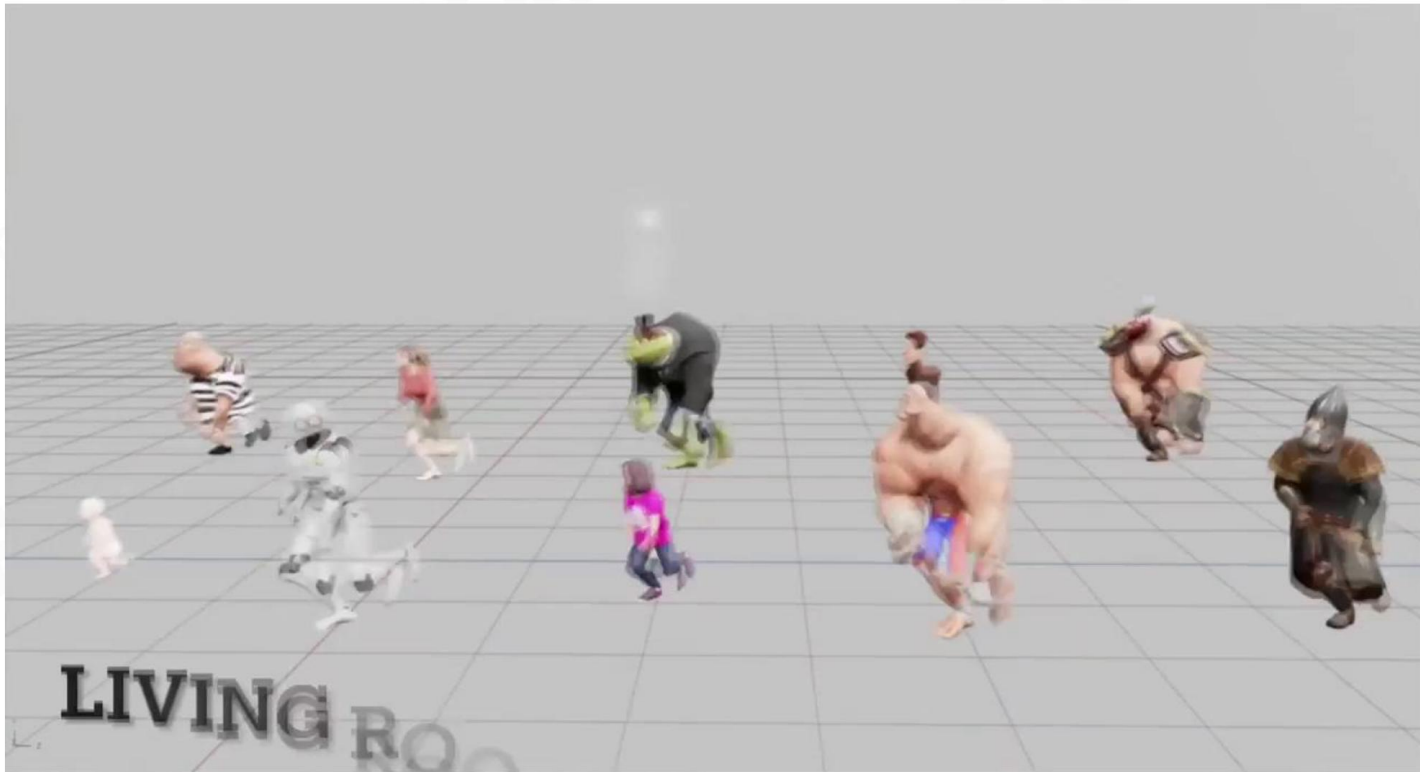
Easy Solution



The solution in Omniverse



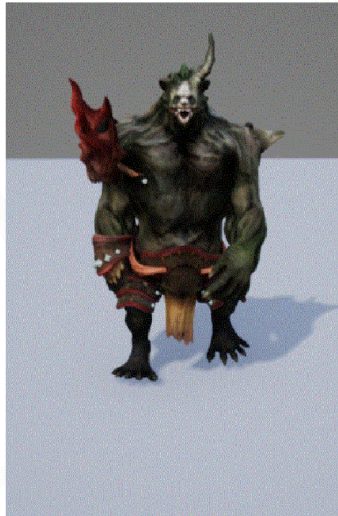
Retargeting Animation in Omniverse



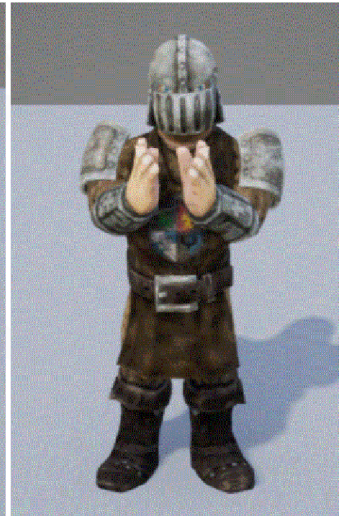
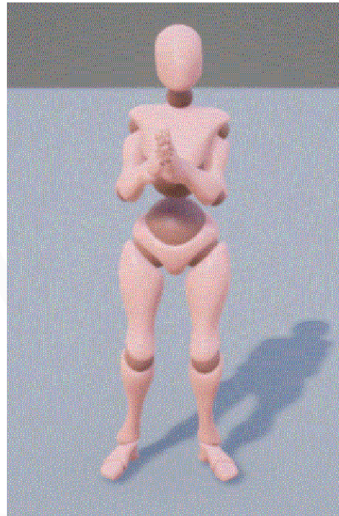


Unresolved Problems of Retargeting

- Self mesh penetration
- Self contact constrains (eg. the hands when clap)
- The balance of the target character



Self mesh penetration



Hands do not contact when clapping



Take Away

- Controlled animation blending system is the key to animate character according to game play
- Inverse Kinematics help character's animation adapt to environment constraints
- Facial expression can be encoded in Action Units in FACS
- Morph target animation is well applied in facial animation
- Retarget can help reuse skeleton animation and facial animations among characters